

**Notes for SPLUNK**

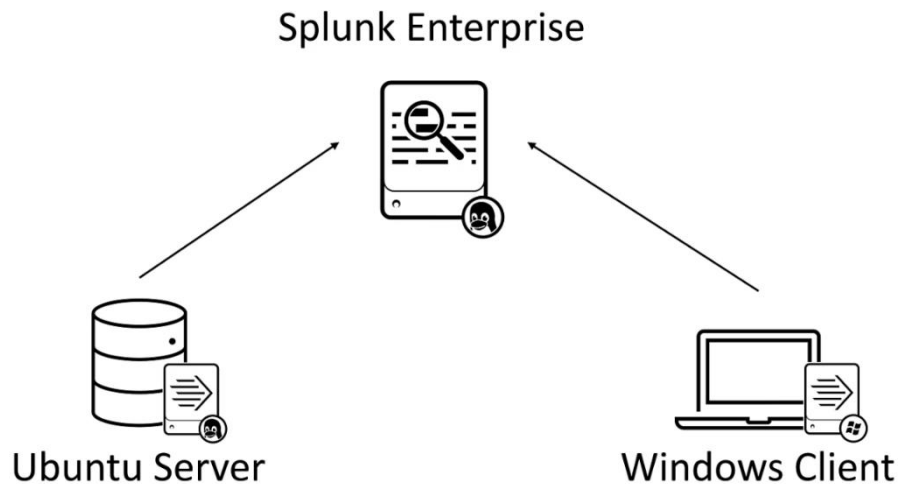
**BY USING HOME LAB**

**KIRAN**

## Contents

|      |                                                                                       |    |
|------|---------------------------------------------------------------------------------------|----|
| I.   | Lab Preparation and how does spluk work:.....                                         | 2  |
| ➤    | <i>Install a Windows universal forwarder</i> .....                                    | 3  |
| ➤    | <i>Install a *nix universal forwarder</i> .....                                       | 3  |
| ➤    | <i>How Data Travels Through Splunk:</i> .....                                         | 3  |
| ➤    | <i>What's an index :</i> .....                                                        | 4  |
| ➤    | <i>splunk config file</i> .....                                                       | 4  |
| ➤    | <i>Enable a Receiver for Splunk Enterprise:</i> .....                                 | 5  |
| ➤    | <i>Shipping Windows Event Logs to Splunk</i> .....                                    | 6  |
| ➤    | <i>Creating a Test Index</i> .....                                                    | 6  |
| II.  | Data Onboarding: .....                                                                | 6  |
| III. | Basic SPL Splunk Processing Language : .....                                          | 7  |
| IV.  | File Lookups:.....                                                                    | 18 |
| V.   | Visualizations and Dashboards: .....                                                  | 24 |
| 1.   | <i>Visualizations:</i> .....                                                          | 24 |
| 2.   | <i>Classic Dashboard</i> .....                                                        | 27 |
| 3.   | <i>building a dynamic Dashboard by token</i> .....                                    | 30 |
| VI.  | Data models: .....                                                                    | 32 |
| VII. | Using Search to Find Suspicious Events: .....                                         | 36 |
| 1.   | <i>HTTP Connections to a Server by IP (Conditional Eval)</i> .....                    | 36 |
| 2.   | <i>Searching 2 Data Sources for a Common Indicator (Using OR)</i> .....               | 37 |
| 3.   | <i>Finding Traces of Look-a-like Domains (Using fuzzylookup)</i> .....                | 38 |
| 4.   | <i>Using Geolocation to Find Anomalous Connections</i> .....                          | 40 |
| 5.   | <i>first Time Login of a User on a Machine</i> .....                                  | 41 |
| 6.   | <i>Identifying Password Guessing Attempts with Failed and Successful Logins</i> ..... | 42 |
| 7.   | <i>Identifying High Network Bandwidth Consumption from a Baseline</i> .....           | 44 |

## I. Lab Preparation and how does splunk work:



In this lab you will learn the installation and configuration of splunk in windows and linux

Search to download splunk and login in the page [splunk.com](https://splunk.com)

**splunk** > a CISCO company

Products ▾ Solutions ▾ Why Splunk? ▾ Resources ▾

Support ▾ 🔍 🌐 👤 **Free Splunk**

### Splunk Enterprise

Download and install Splunk Enterprise trial on your own hardware or cloud instance so you can collect, analyze, visualize and act on all your data — no matter its source. Try indexing up to 500MB/day for 60 days, no credit card required.

[Get My Free Trial](#) [View Product](#)

**Windows** Linux Mac OS

**64-bit** **Windows 10**  
**Windows Server 2019, 2022**

.msi 509.86 MB [Download Now](#)

**Windows** **Linux** Mac OS

**64-bit** **3.x+, 4.x+, or 5.4.x kernel Linux distributions**

|      |           |                              |
|------|-----------|------------------------------|
| .deb | 520.37 MB | <a href="#">Download Now</a> |
| .tgz | 679.42 MB | <a href="#">Download Now</a> |
| .rpm | 679.24 MB | <a href="#">Download Now</a> |

We download the version of splunk of system we will use ( I use my windows host to make splunk work better , but you can use any system ( windows , linux or macOS) in VM or host , there are no different

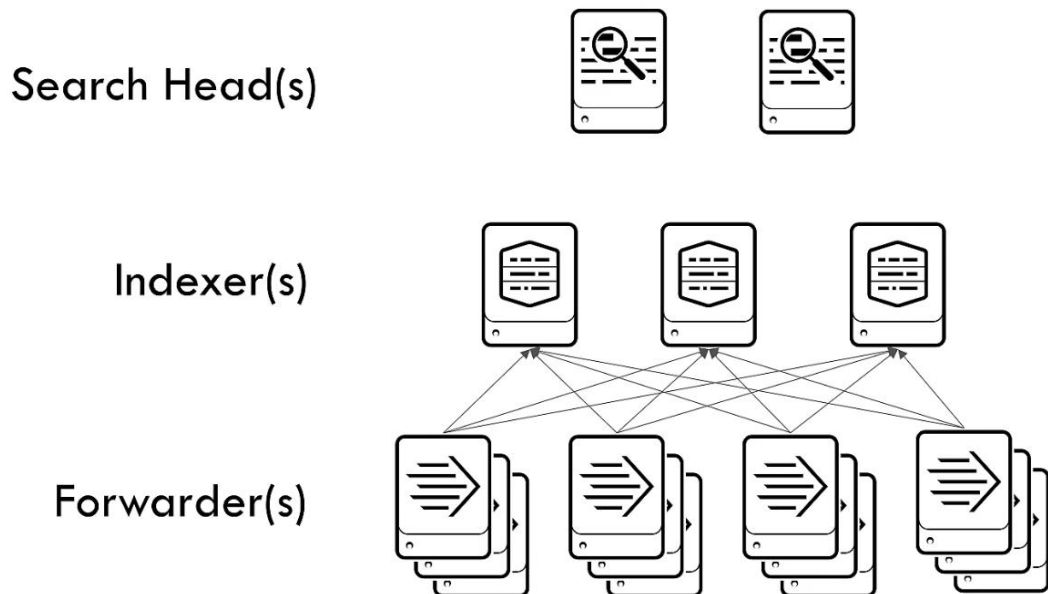
➤ **Install a Windows universal forwarder**

<https://docs.splunk.com/Documentation/Forwarder/9.2.0/Forwarder/InstallWindowsuniversalforwarderfromaninstaller#:~:text=Download%20the%20universal%20forwarder%20from,Splunk%20Enterprise%20or%20Splunk%20Cloud.>

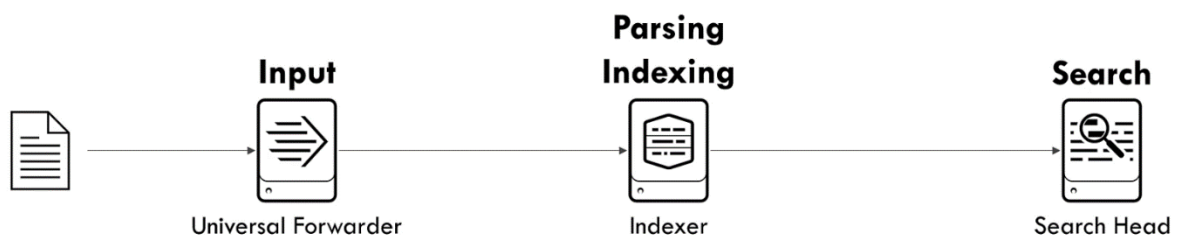
➤ **Install a \*nix universal forwarder**

<https://docs.splunk.com/Documentation/Forwarder/9.2.0/Forwarder/InstallLinuxuniversalforwarder>

**typical architecture of splunk :**

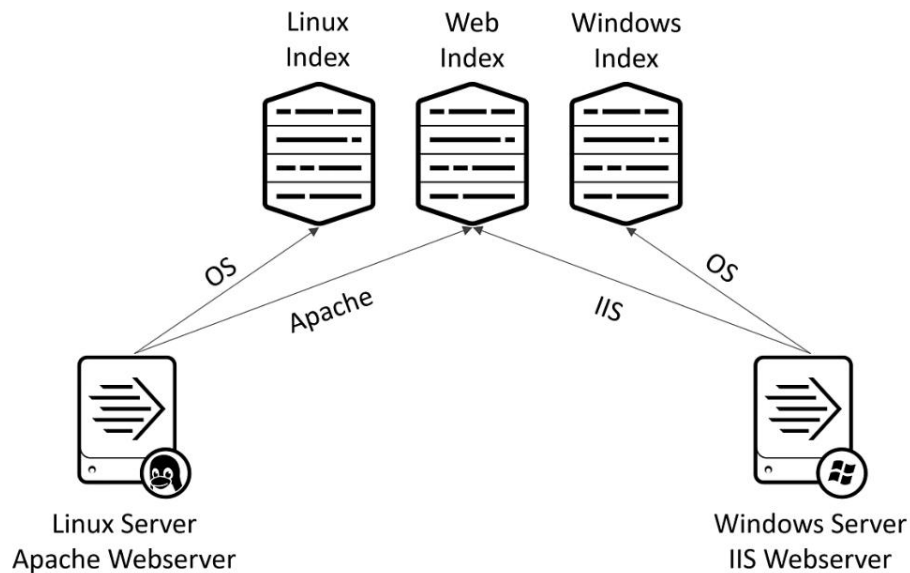


➤ **How Data Travels Through Splunk:**

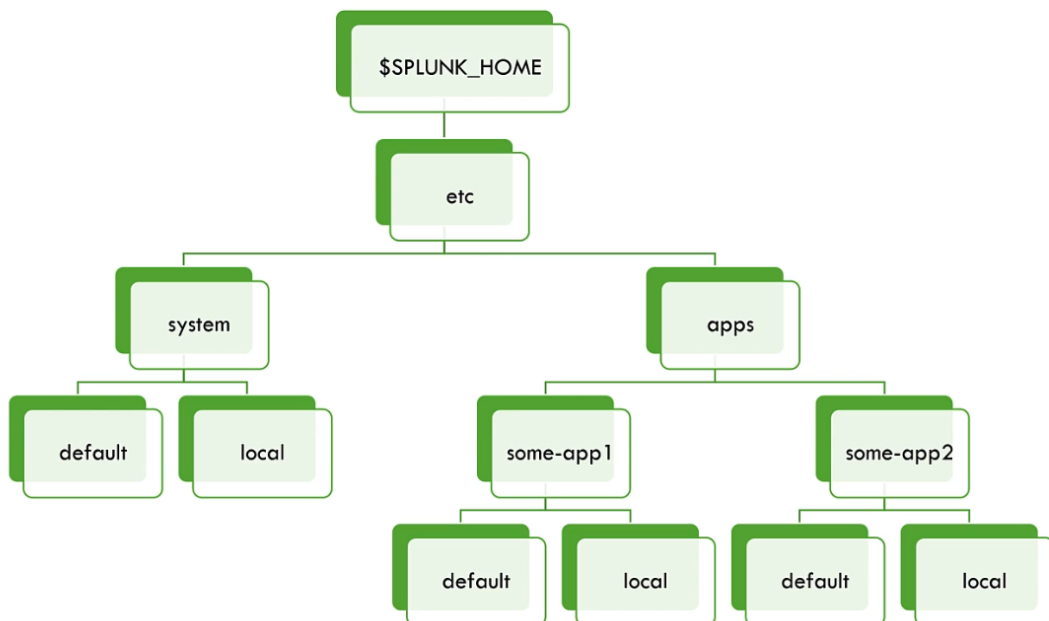


➤ **What's an index :**

an index is a repository or storage location where Splunk stores and manages data. When data is ingested into Splunk, it is indexed and stored in these index locations. Indexes serve as the foundation for searching, analyzing, and visualizing data within Splunk



➤ **splunk config file**



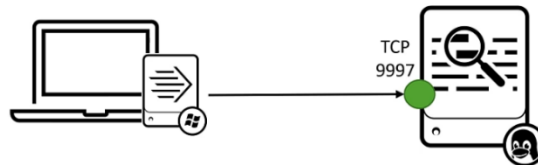
the config file in linux and windows are the same way

but root splunk folder in linux : /opt/splunk

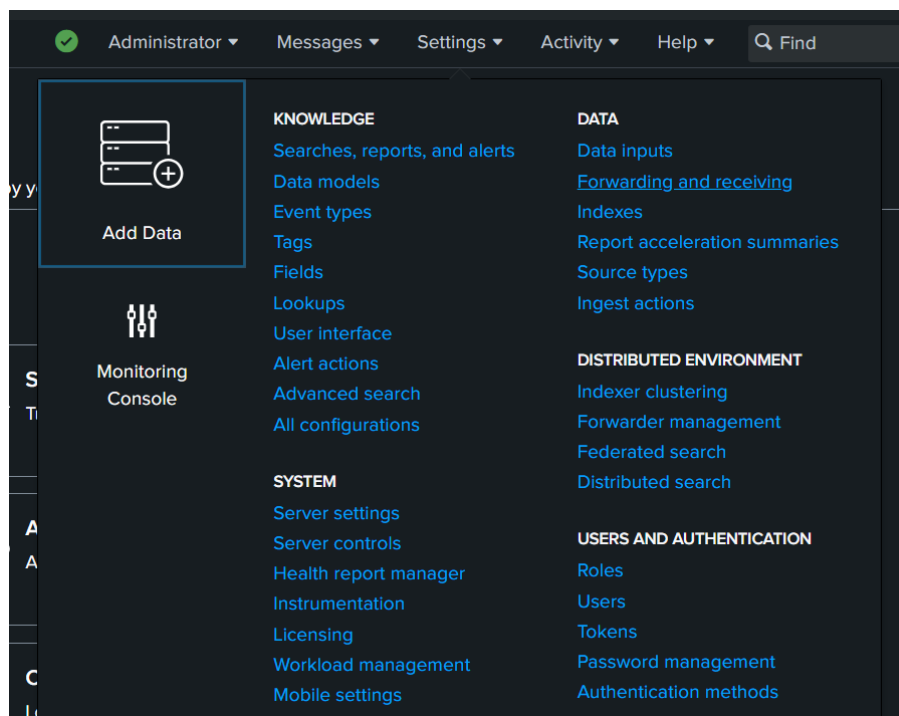
and window : C:\Program Files\Splunk

➤ **Enable a Receiver for Splunk Enterprise:**

- ❑ Receivers are the ports where Splunk accepts data coming from Forwarders
- ❑ It is possible to configure multiple receivers, but normally there is one
- ❑ The default TCP port for receiving is 9997 or, in case of TLS protected connections 9998



go to setting in splunk enterprise :



we click on forwarding and receiving then receive data :

The screenshot shows the 'Configure receiving' dialog box. The title is 'Configure receiving'. Below the title, it says 'Set up this Splunk instance to receive data from forwarder(s)'. There is a text input field labeled 'Listen on this port \*' with the value '9997' entered. Below the input field, a note states 'For example, 9997 will receive data on TCP port 9997.' At the bottom right of the dialog, there are two buttons: 'Cancel' and 'Save'.

We input the port that we want and save the settings

➤ **Shipping Windows Event Logs to Splunk**

<https://docs.splunk.com/Documentation/Splunk/9.2.1/Data/MonitorWindowseventlogdata#:~:text=To%20monitor%20Windows%20Event%20Log,data%20into%20Splunk%20Cloud%20Platform.>

➤ **Creating a Test Index**

<https://docs.splunk.com/Documentation/Splunk/9.2.0/Data/Useatestindex>

## **II. Data Onboarding:**

<https://docs.splunk.com/Documentation/Splunk/latest/Deploy/Distributedoverview>

<https://docs.splunk.com/Documentation/Splunk/latest/DistSearch/Whatisdistributedsearch>

<https://docs.splunk.com/Documentation/Splunk/latest/Deploy/Datapipeline>

<https://docs.splunk.com/Documentation/Splunk/latest/Indexer/Aboutindexesandindexers>

<https://docs.splunk.com/Documentation/Splunk/latest/Indexer/Aboutmanagingindexes>

<https://docs.splunk.com/Documentation/Splunk/latest/Forwarding/Enableareceiver>

<https://docs.splunk.com/Documentation/Splunk/latest/Admin/Aboutconfigurationfiles>

<https://docs.splunk.com/Documentation/Splunk/latest/Admin/Wheretofindtheconfigurationfiles>

<https://docs.splunk.com/Documentation/Splunk/latest/Admin/Listofconfigurationfiles>

<https://docs.splunk.com/Documentation/Splunk/latest/Admin/Whatsanapp>

<https://docs.splunk.com/Documentation/Splunk/latest/Data/MonitorWindowseventlogdata>

### III. Basic SPL Splunk Processing Language :

Botv2: A sample security dataset and CTF platform for information security professionals, researchers, students, and enthusiasts. This page hosts information regarding the version 2 *dataset*

To download : <https://github.com/splunk/botv2>

How to download : <https://edscopybersec.com/posts/installing-splunk-botv2-dataset/>

The same way in linux and windows

➤ Basic search components :

```
index=main sourcetype="stream:http" bob  
| stats count by http_user_agent
```

- ☐ Search terms
- ☐ Pipes
- ☐ Commands
- ☐ Functions
- ☐ Clauses



➔ index=botsv2 earliest=0 sourcetype="stream:http" | stats count by host

**Nouvelle recherche**

1 index=botsv2 earliest=0 sourcetype="stream:http"  
2 | stats count by host

24 dernières heures

Sur 1090 922 événement, 1090 922 qui correspond

Aucun échantillon d'événement

Tâche

Mode Verbeux

Événements (1090 922) Patterns **Statistiques (7)** Visualisation

20 par page Format Aperçu

| host       | count  |
|------------|--------|
| altair     | 19822  |
| cassiopeia | 43165  |
| eridanus   | 50266  |
| gacrux     | 464155 |
| jabbah     | 208860 |
| jupiter    | 279635 |

this command use for count the satastics by host from the index botsv2 source stream:http

➔ index=botsv2 earliest=0 sourcetype="stream:http" | eval src\_ip\_port = src\_ip.".".src\_port

**Nouvelle recherche**

1 index=botsv2 earliest=0 sourcetype="stream:http"  
2 | eval src\_ip\_port = src\_ip.".".src\_port

24 dernières heures

Sur 894 423 événement, 894 423 qui correspond

Événements (894 423) Patterns Statistiques Visualisation

Mettre en forme la chronologie

< Masquer les champs

CHAMPS SÉLECTIONNÉS

- a host 7
- a source 2
- a sourcetype 1
- a src\_ip\_port 100+
- # status 18

CHAMPS INTÉRESSANTS

**src\_ip\_port**

>100 Valeurs, 85,46 % des événements

Sélectionné Oui Non

Rapports

Top valeurs Top valeurs par heure Valeurs rares

Événements avec ce champ

| 10 premières valeurs | Nombre | %       |
|----------------------|--------|---------|
| 10.0.2.101:57456     | 8 261  | 1,081 % |
| 10.0.2.101:57466     | 6 637  | 0,868 % |
| 10.0.1.120:8014      | 3 683  | 0,482 % |
| 10.0.2.107:62365     | 3 624  | 0,474 % |
| 10.0.2.101:57436     | 3 220  | 0,421 % |
| 10.0.2.107:62182     | 2 544  | 0,333 % |
| 10.0.2.105:53755     | 2 520  | 0,33 %  |
| 10.0.2.107:62174     | 2 519  | 0,33 %  |
| 10.0.2.107:62181     | 2 088  | 0,273 % |
| 10.0.2.107:62175     | 1 840  | 0,241 % |

cookie: image=translation=storage=%/B%7D; image=translation=tile-version=%7B%7D; form key=m4dSsC4sPy

command eval use to create a new field by combinations of another field

➔ `index=botsv2 earliest=0 sourcetype="stream:http" | table src_ip, dest_ip, dest_port`

| src_ip               | dest_ip    | dest_port |
|----------------------|------------|-----------|
| 253.220.103.168      | 172.31.7.2 | 80        |
| 253.220.103.168      | 172.31.7.2 | 80        |
| 253.220.103.168      | 172.31.7.2 | 80        |
| 253.220.103.168      | 172.31.7.2 | 80        |
| 253.220.103.168      | 172.31.7.2 | 80        |
| 253.220.103.168      | 172.31.7.2 | 80        |
| localhost:8000/fr-FR | 172.31.7.2 | 80        |

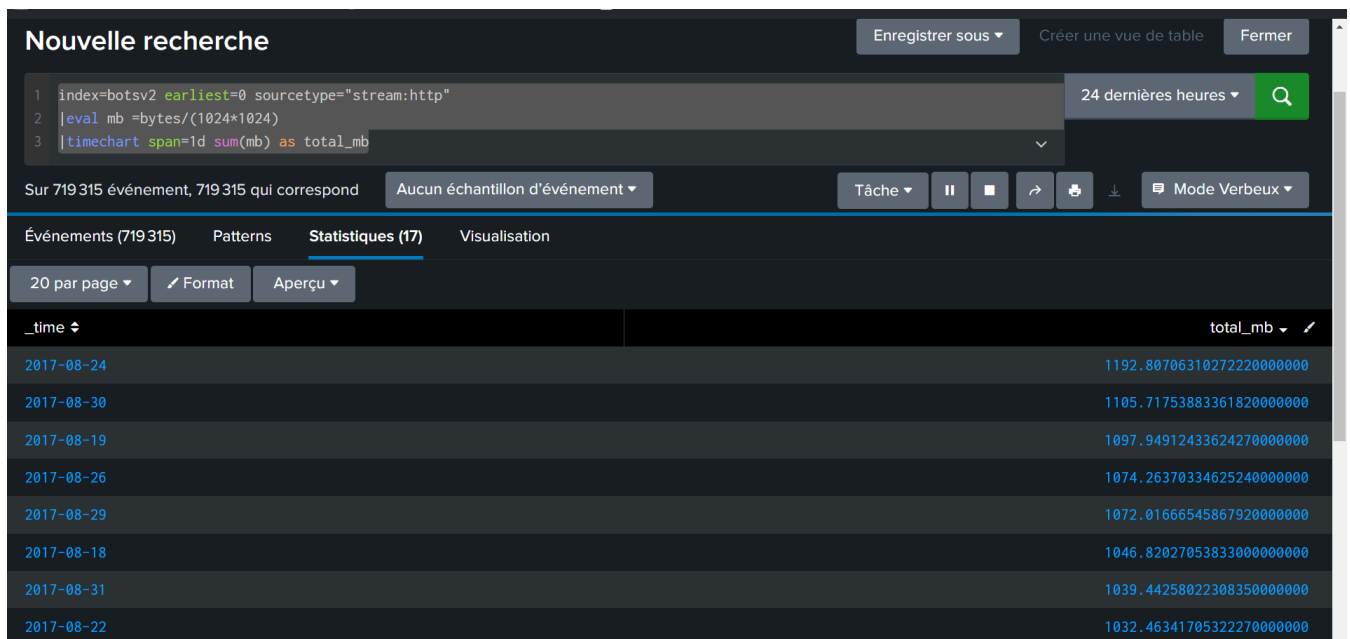
Command **table** use to create table of the fields those we are choosing

➔ `index=botsv2 earliest=0 sourcetype="stream:http"`  
`| stats dc(src_port) as number_of_connection by src_ip, dest_ip, dest_port`  
`| table src_ip dest_ip , number_of_connection`

| src_ip       | dest_ip         | number_of_connection |
|--------------|-----------------|----------------------|
| 172.31.10.10 | 169.254.169.254 | 5432                 |
| 45.77.65.211 | 172.31.4.249    | 4854                 |
| 10.0.2.110   | 10.0.1.120      | 2407                 |
| 10.0.2.107   | 10.0.1.120      | 2338                 |
| 10.0.2.103   | 10.0.1.120      | 2248                 |

Function **dc** calculates the number of distinct source ports (**src\_port**) for each combination of **src\_ip**, **dest\_ip**, and **dest\_port**

➔ `index=botsv2 earliest=0 sourcetype="stream:http"`  
`|eval mb =bytes/(1024*1024)`  
`|timechart span=1d sum(mb) as total_mb`

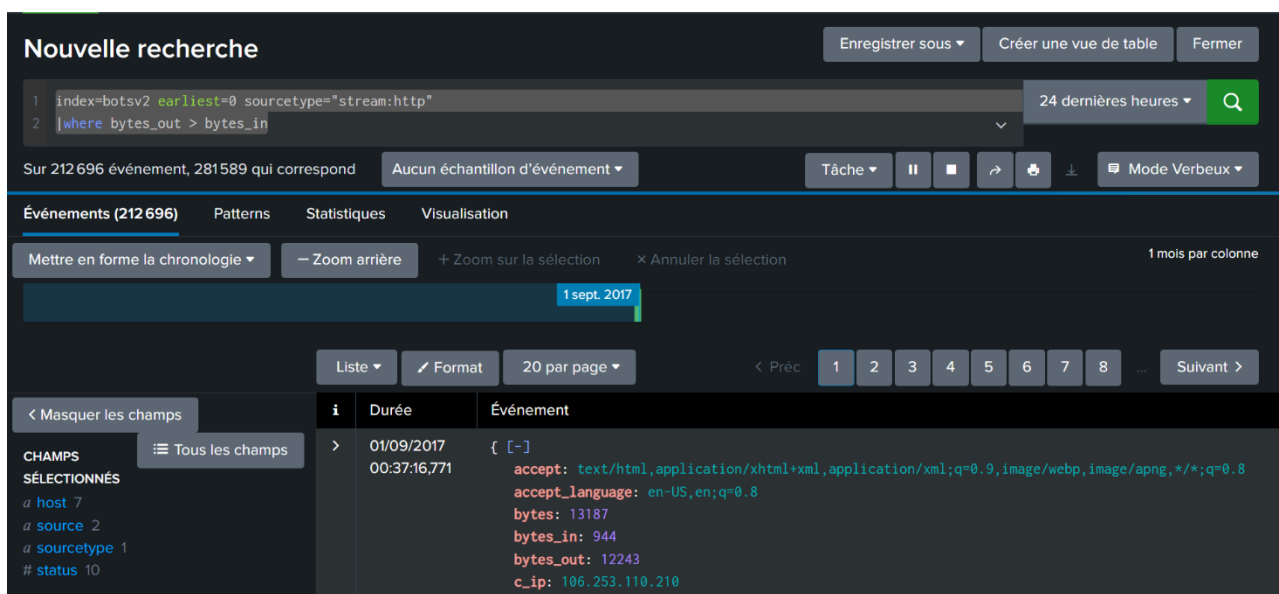


**eval mb = bytes/(1024\*1024):** Calculates the total megabytes transferred by dividing the **bytes** field by 1024\*1024.

**timechart span=1d sum(mb) as total\_mb:** Aggregates the data over each day (**span=1d**) and sums up the total megabytes transferred (**sum(mb)**), creating a timechart.

We can use with **span** function s,m,h,d,w,mon and y for specific the duration of time

➔ `index=botsv2 earliest=0 sourcetype="stream:http" |where bytes_out > bytes_in`



**where bytes\_out > bytes\_in:** Filters events where the number of bytes sent out (**bytes\_out**) is greater than the number of bytes received (**bytes\_in**).

➔ `index=botsv2 earliest=0 sourcetype="stream:http"`  
`| rex field=http_user_agent "Windows\s+NT\s+(?<win_nt_version>[\d\.]+)"`

Sur 644 246 événement, 644 246 qui corre:

Événements (644 246) Patterns S

Mettre en forme la chronologie - 2

< Masquer les champs

CHAMPS SÉLECTIONNÉS

- `a host` 7
- `a source` 2
- `a sourcetype` 1
- `# status` 18
- `# win_nt_version` 6

CHAMPS INTÉRESSANTS

`win_nt_version`

6 Valeurs, 29,488 % des événements

Sélectionné Oui Non

Rapports

Moyenne sur la durée Valeur maximum sur la durée Valeur minimum sur la durée

Top valeurs Top valeurs par heure Valeurs rares

Événements avec ce champ

Moy: 6.141569070471528 Min: 5.1 Max: 10.0 Std Dév: 0.4891594676686107

| Valeurs | Nombre  | %        |
|---------|---------|----------|
| 6.1     | 148 228 | 79,569 % |
| 6.333   | 25 443  | 13,658 % |
| 5.1     | 7 549   | 4,052 %  |
| 10.0    | 2 360   | 1,267 %  |
| 6.3     | 1 438   | 0,772 %  |
| 6.0     | 1 270   | 0,682 %  |

1 mois par colonne

6 7 8 ... Suivant >

image/webp,image/apng,\*/\*;q=0.8

e-version=%7B%7D; form\_key=m4dSsC4sP

dest\_content: <doctype html>

`rex field=http_user_agent "Windows\s+NT\s+(?<win_nt_version>[\d\.]+)"`: Uses regular expression to extract the version of Windows NT from the `http_user_agent` field and captures it into a new field called `win_nt_version`. The regular expression `Windows\s+NT\s+(?<win_nt_version>[\d\.]+)` matches "Windows", followed by "NT", then captures the version number consisting of digits and periods (`[\d\.]+`).

This query should help you extract and identify the version of Windows NT from the user-agent strings in your HTTP stream events.

➔ `index=botsv2 earliest=0 sourcetype="stream:http"`  
`| stats sum(bytes_in) AS bytes_in, sum(bytes_out) AS bytes_out BY src_ip, dest_ip`  
`| eval mbytes_in = round(bytes_in/pow(1024,2),2), mbytes_out =`  
`round(bytes_out/pow(1024,2),2) | fields - bytes_in, bytes_out`  
`| stats avg(mbytes_in) AS avg_mbytes_in, avg(mbytes_out) AS avg_mbytes_out`

The screenshot shows the Splunk Enterprise search interface. At the top, there's a navigation bar with 'splunk>enterprise' and various menu items like 'Apps', 'Administrator', 'Messages', 'Paramètres', 'Activité', 'Aide', and 'Rechercher'. Below this is a secondary bar with 'Recherche', 'Analytics', 'Jeux de données', 'Rapports', 'Alertes', and 'Tableaux de bord'. The main section is titled 'Nouvelle recherche' (New search). A search bar contains the following query:

```
1 | index=botsv2 earliest=0 sourcetype="stream:http"
2 | stats sum(bytes_in) AS bytes_in, sum(bytes_out) AS bytes_out BY src_ip, dest_ip
3 | eval mbytes_in = round(bytes_in/pow(1024,2),2), mbytes_out = round(bytes_out/pow(1024,2),2)
4 | fields - bytes_in, bytes_out
5 | stats avg(mbytes_in) AS avg_mbytes_in, avg(mbytes_out) AS avg_mbytes_out
6 | sort - mbytes_in
```

Below the search bar, it indicates '1408636 événement (25/03/2024 00:00:00,000 à 26/03/2024 00:43:27,000)'. There are buttons for 'Enregistrer sous', 'Créer une vue de table', and 'Fermer'. The results are displayed in a table with two columns: 'avg\_mbytes\_in' and 'avg\_mbytes\_out'. The first row shows values '0.5358867125463284' and '14.524282689253472'.

**| eval mbytes\_in = round(bytes\_in/pow(1024,2),2), mbytes\_out = round(bytes\_out/pow(1024,2),2):** Converts the sum of bytes into megabytes and rounds the result to two decimal places.

**| fields - bytes\_in, bytes\_out:** Removes the original **bytes\_in** and **bytes\_out** fields, leaving only the **mbytes\_in** and **mbytes\_out** fields.

**| stats avg(mbytes\_in) AS avg\_mbytes\_in, avg(mbytes\_out) AS avg\_mbytes\_out:** Calculates the average megabytes transferred for each source-destination IP pair.

**| sort - mbytes\_in:** Sorts the results based on the average megabytes transferred in descending order.

This query provides insights into the average megabytes transferred between source and destination IP pairs, sorted by the average megabytes in descending order.

➔ `index=botsv2 earliest=0 sourcetype="stream:http" | stats list(http_method) AS http_methods`

The screenshot shows the Splunk Enterprise search interface. The search bar contains the query: `index=botsv2 earliest=0 sourcetype="stream:http" | stats list(http_method) AS http_methods`. The search results show 294,089 events. The results table has a single column 'http\_methods' with values: GET, GET, GET, GET, GET, GET, GET, GET, GET, GET.

**stats list(http\_method) AS http\_methods:** Calculates a list of unique values of the **http\_method** field across all events and assigns it to a new field named **http\_methods**.

This query will give you a list of unique HTTP methods present in the dataset.

➔ `index=botsv2 earliest=0 sourcetype="stream:http" | stats values(http_method) AS http_methods`

The screenshot shows the Splunk Enterprise search interface. The search bar contains the query: `index=botsv2 earliest=0 sourcetype="stream:http" | stats values(http_method) AS http_methods`. The search results show 212,401 events. The results table has a single column 'http\_methods' with values: GET, HEAD, OPTIONS, POST, PROPFIND, PUT.

**stats values(http\_method) AS http\_methods:** Calculates a list of unique values of the **http\_method** field across all events and assigns it to a new field named **http\_methods**.

➔ `index=botsv2 earliest=0 sourcetype="stream:http"`  
`| eval ua_length = len(http_user_agent)`  
`| stats min(ua_length) AS min, max(ua_length) AS max, avg(ua_length) AS avg`

The screenshot shows the Splunk Enterprise interface. The search bar contains the query: `index=botsv2 earliest=0 sourcetype="stream:http" | eval ua_length = len(http_user_agent) | stats min(ua_length) AS min, max(ua_length) AS max, avg(ua_length) AS avg`. The results are displayed in a table with three columns: `min`, `max`, and `avg`. The values are 7, 211, and 101.62527590448688 respectively. The interface also shows a search bar, navigation tabs, and a sidebar.

- **eval ua\_length = len(http\_user\_agent)**: Calculates the length of the **http\_user\_agent** field for each event and assigns it to a new field named **ua\_length**.
- **stats min(ua\_length) AS min, max(ua\_length) AS max, avg(ua\_length) AS avg**: Calculates the minimum, maximum, and average length of the **http\_user\_agent** field across all events.

This query will provide you with statistics regarding the length of the **http\_user\_agent** field, including the minimum, maximum, and average lengths.

➔ `index=botsv2 earliest=0 sourcetype="stream:http" | eval ua_length = len(http_user_agent)`  
`| eventstats min(ua_length) AS min, max(ua_length) AS max, avg(ua_length) AS avg`  
`| where ua_length=min OR ua_length=max`

The screenshot shows the Splunk Enterprise interface. The search bar contains the query: `index=botsv2 earliest=0 sourcetype="stream:http" | eval ua_length = len(http_user_agent) | eventstats min(ua_length) AS min, max(ua_length) AS max, avg(ua_length) AS avg | where ua_length=min OR ua_length=max`. The results are displayed in a table with three columns: `min`, `max`, and `avg`. The values are 7, 211, and 101.62527590448688 respectively. The interface also shows a search bar, navigation tabs, and a sidebar.

`| eventstats min(ua_length) AS min, max(ua_length) AS max, avg(ua_length) AS avg`: Uses **eventstats** to compute the minimum, maximum, and average length of the **http\_user\_agent** field across all events.

```
➔ index=botsv2 earliest=0 sourcetype="stream:http" | streamstats count | table count, _raw
```

[illegible]

- | **streamstats count:** Uses **streamstats** to calculate the count of events encountered so far.

This query will give you a table showing the count of events encountered so far along with the corresponding raw event data.

➔ `index=botsv2 earliest=0 sourcetype="stream:http" | chart count BY http_method host`

Nouvelle recherche

Enregistrer sous

Cr  er une vue de table

Fermer

```
1 index=botsv2 earliest=0 sourcetype="stream:http"
2 | chart count BY http_method host
```

24 derni  res heures

Sur 619 231   v  nement, 619 231 qui correspond

Aucun   chantillon d'  v  nement

T  che

II

Mode Verbeux

  v  nements (619 231)

Patterns

Statistiques (6)

Visualisation

20 par page

Format

Aper  u

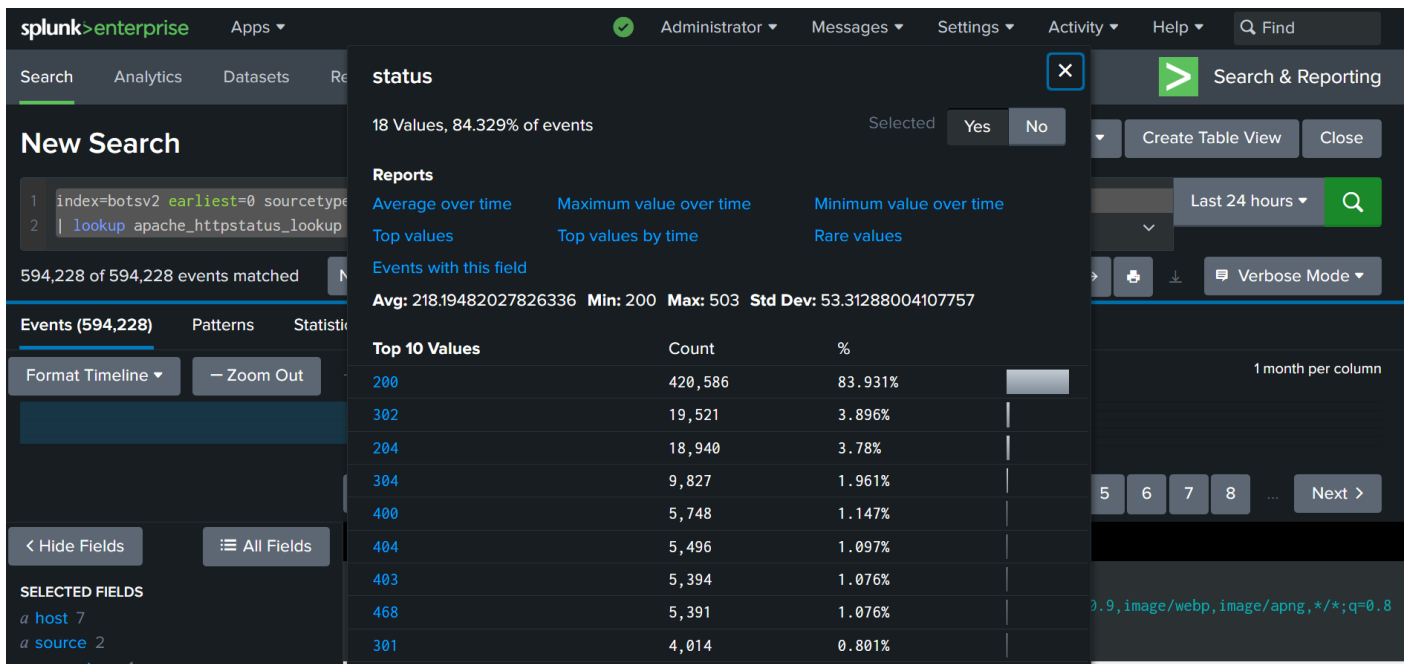
| http_method | altair | cassiopeia | eridanus | gacrux | jabbah | jupiter | matar |
|-------------|--------|------------|----------|--------|--------|---------|-------|
| GET         | 10718  | 19801      | 26981    | 172529 | 100342 | 127474  | 1     |
| HEAD        | 0      | 0          | 106      | 0      | 0      | 2714    | 0     |
| OPTIONS     | 0      | 0          | 0        | 0      | 0      | 6       | 0     |
| POST        | 0      | 1440       | 0        | 19525  | 1052   | 15722   | 0     |
| PROPFIND    | 0      | 0          | 0        | 0      | 0      | 3       | 0     |
| PUT         | 0      | 0          | 0        | 0      | 129    | 0       | 0     |

**| chart count BY http\_method host:** This part of the query utilizes the **chart** command to create a chart where the x-axis represents the unique values of the **http\_method** field, the y-axis represents the count of events for each method, and each series in the chart represents a different value of the **host** field.

This chart provides a visual representation of the distribution of HTTP methods across different hosts in your dataset. It allows you to quickly analyze which HTTP methods are more commonly used on which hosts.



→ `index=botsv2 earliest=0 sourcetype="stream:http"`  
 | `lookup apache_httpstatus_lookup status as http_status`

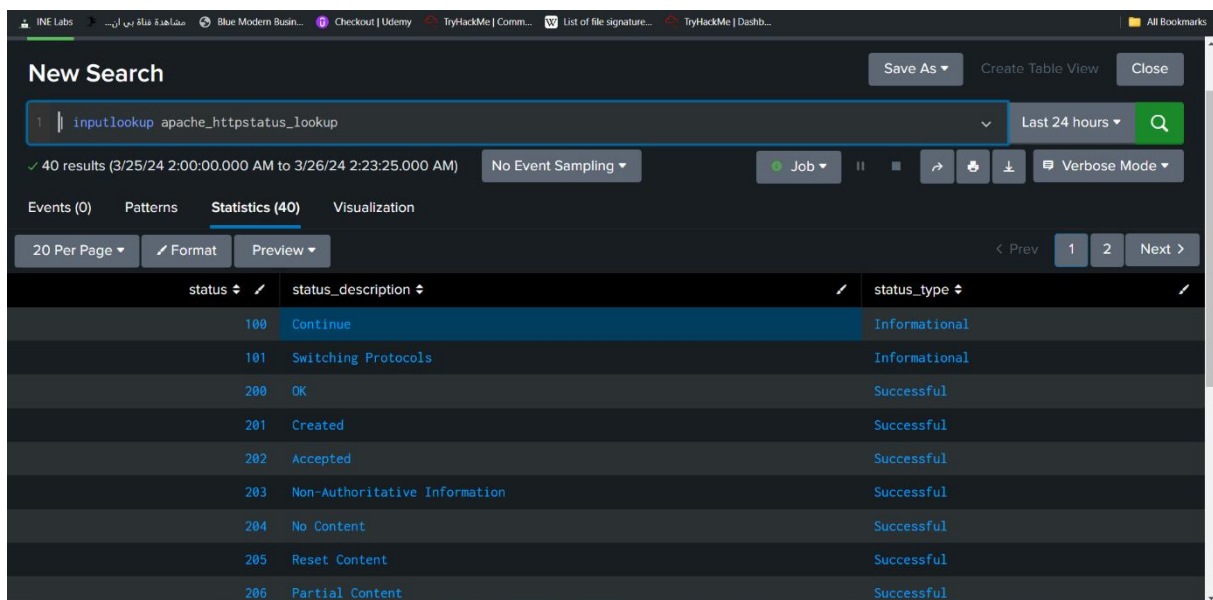


The screenshot shows the Splunk Enterprise interface. On the left, the 'New Search' panel displays the search query: `index=botsv2 earliest=0 sourcetype="stream:http" | lookup apache_httpstatus_lookup status as http_status`. It indicates that 594,228 of 594,228 events matched. The main panel shows a table of results for the `status` field. The table has three columns: `status`, `Count`, and `%`. The results are as follows:

| status | Count   | %       |
|--------|---------|---------|
| 200    | 420,586 | 83.931% |
| 302    | 19,521  | 3.896%  |
| 204    | 18,940  | 3.78%   |
| 304    | 9,827   | 1.961%  |
| 400    | 5,748   | 1.147%  |
| 404    | 5,496   | 1.097%  |
| 403    | 5,394   | 1.076%  |
| 468    | 5,391   | 1.076%  |
| 301    | 4,014   | 0.801%  |

| `lookup apache_httpstatus_lookup status as http_status`: Performs a lookup operation using the `apache_httpstatus_lookup` table on the `http_status` field in your events. It attempts to match the values of the `http_status` field in your events with the values of the `status` field in the lookup table.

→ | `inputlookup apache_httpstatus_lookup`



The screenshot shows the Splunk Enterprise interface. On the left, the 'New Search' panel displays the search query: `inputlookup apache_httpstatus_lookup`. It indicates that 40 results were found. The main panel shows a table of results for the `status` field. The table has three columns: `status`, `status_description`, and `status_type`. The results are as follows:

| status | status_description            | status_type   |
|--------|-------------------------------|---------------|
| 100    | Continue                      | Informational |
| 101    | Switching Protocols           | Informational |
| 200    | OK                            | Successful    |
| 201    | Created                       | Successful    |
| 202    | Accepted                      | Successful    |
| 203    | Non-Authoritative Information | Successful    |
| 204    | No Content                    | Successful    |
| 205    | Reset Content                 | Successful    |
| 206    | Partial Content               | Successful    |

| `inputlookup apache_httpstatus_lookup`: This command reads the contents of the lookup table named `apache_httpstatus_lookup` and presents the data as search results.

This command is useful for viewing the contents of a lookup table directly within your search interface, which can be helpful for debugging or inspecting the data in the lookup table.

```
| inputlookup apache_httpstatus_lookup
| append
  [ makeresults
    | eval status=418, status_description="I'm a teapot", status_type="Client Error" ]
| fields - _time
| outputlookup apache_httpstatus_lookup
```

The screenshot shows the Splunk Enterprise web interface. At the top, there's a navigation bar with 'splunk>enterprise' and various menu items like 'Apps', 'Administrator', 'Messages', 'Settings', 'Activity', and 'Help'. Below this is a search bar and a 'Search & Reporting' button. The main section is titled 'New Search' and contains a search query in a text area:

```
1 | inputlookup apache_httpstatus_lookup
2 | append
3   [ makeresults
4     | eval status=418, status_description="I'm a teapot", status_type="Client Error" ]
5 | fields - _time
6 | outputlookup apache_httpstatus_lookup
```

Below the query, it shows '41 results (3/25/24 2:00:00.000 AM to 3/26/24 2:43:12.000 AM)' and a 'No Event Sampling' dropdown. There are also buttons for 'Job', 'Verbose Mode', and a 'Save As' dropdown. The results are displayed in a table with columns 'status', 'status\_description', and 'status\_type'.

| status | status_description  | status_type   |
|--------|---------------------|---------------|
| 100    | Continue            | Informational |
| 101    | Switching Protocols | Informational |
| 200    | OK                  | Successful    |

**| append [ makeresults | eval status=418, status\_description="I'm a teapot", status\_type="Client Error" ]:** This appends a new entry to the results retrieved from the **apache\_httpstatus\_lookup** table. The new entry has a status of 418, a status description of "I'm a teapot", and a status type of "Client Error".

**| fields - \_time:** This command removes the **\_time** field from the events, as it's not necessary for the lookup table.

**| outputlookup apache\_httpstatus\_lookup:** Finally, this command writes the combined results (original lookup table plus the new entry) back to the **apache\_httpstatus\_lookup** lookup table.

In summary, this query adds a new entry to the **apache\_httpstatus\_lookup** lookup table with the status code 418, a custom status description, and a status type. It then updates the lookup table with this new entry.

## IV. File Lookups:

### 1. CSV file Lookups

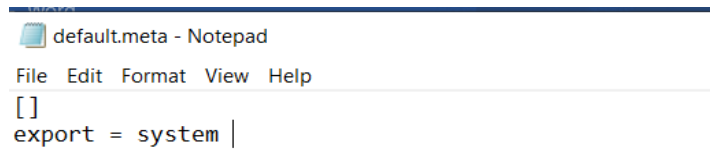
- ⇒ file located in the search head
- ⇒ lives either in the app's lookups folder or /etc/system/lookups

In cmd : ~/splunk/etc/apps

mkdir well\_known\_ports

mkdir metadata

notpade default.meta

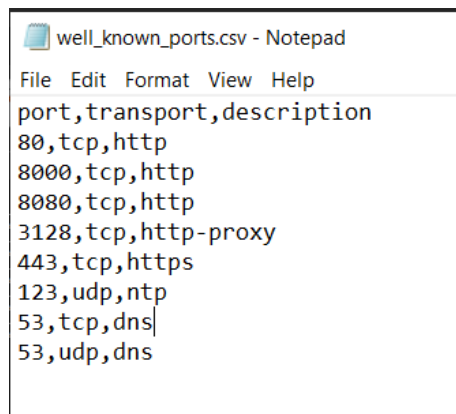


```
default.meta - Notepad
File Edit Format View Help
[]
export = system |
```

mkdir lookups

cd lookups

notepad well\_known\_ports.csv



```
well_known_ports.csv - Notepad
File Edit Format View Help
port,transport,description
80,tcp,http
8000,tcp,http
8080,tcp,http
3128,tcp,http-proxy
443,tcp,https
123,udp,ntp
53,tcp,dns
53,udp,dns
```

mkdir default

cd default

notepad transforms.conf

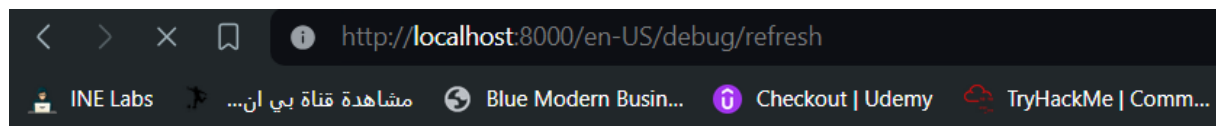
transforms.conf - Notepad

File Edit Format View Help

```
[well_known_ports]
```

```
filename = well_known_ports.csv
```

<http://localhost:8000/en-US/debug/refresh>



Refresh

➔ index=botsv2 sourcetype="pan:traffic" earliest=0  
| lookup well\_known\_ports port AS dest\_port, transport  
| table src\_ip, dest\_ip, dest\_port, transport, description

The screenshot shows the Splunk Search interface. At the top, there's a navigation bar with 'Search', 'Analytics', 'Datasets', 'Reports', 'Alerts', and 'Dashboards'. A 'Search & Reporting' button is on the right. Below the navigation bar, the 'New Search' section contains a search bar with the following query:

```
1 index=botsv2 sourcetype="pan:traffic" earliest=0
2 | lookup well_known_ports port AS dest_port, transport
3 | table src_ip, dest_ip, dest_port, transport, description
```

Below the search bar, it indicates '1,862,616 events (before 3/26/24 5:28:41.000 AM)' and 'No Event Sampling'. There are buttons for 'Job', 'Verbose Mode', and a 'Save As' button. Below this, there are tabs for 'Events (1,862,616)', 'Patterns', 'Statistics (1,862,616)', and 'Visualization'. The 'Statistics' tab is selected. At the bottom, there's a table with 5 columns: 'src\_ip', 'dest\_ip', 'dest\_port', 'transport', and 'description'. The table shows 10 rows of data.

| src_ip     | dest_ip       | dest_port | transport | description |
|------------|---------------|-----------|-----------|-------------|
| 10.0.2.101 | 10.0.1.100    | 53        | udp       | dns         |
| 10.0.2.107 | 10.0.1.120    | 8014      | tcp       |             |
| 10.0.2.109 | 10.0.1.100    | 445       | tcp       |             |
| 10.0.2.101 | 75.98.70.160  | 80        | tcp       | http        |
| 10.0.1.100 | 8.8.8.8       | 53        | udp       | dns         |
| 10.0.1.200 | 52.40.10.231  | 443       | tcp       | https       |
| 10.0.2.101 | 23.63.227.171 | 80        | tcp       | http        |
| 10.0.2.101 | 10.0.1.120    | 8014      | tcp       |             |
| 10.0.2.101 | 23.63.227.171 | 80        | tcp       | http        |
| 10.0.1.100 | 8.8.8.8       | 53        | udp       | dns         |

| lookup well\_known\_ports port AS dest\_port, transport: Performs a lookup operation using the well\_known\_ports lookup table. It matches the dest\_port field from the events with the port field in the well\_known\_ports table and retrieves the corresponding transport field.

This query helps to enrich the traffic events with additional information about the destination ports, including the transport protocol and a description of the service associated with each port

## 2. KV Store Lookups:

- ⇒ Splunk comes with a instance of MongoDB
- ⇒ Great large of lookups or when data is updated often

In cmd : ~/splunk/etc/apps/well\_known\_ports/default:

notepad collections.conf

collections.conf - Notepad

```
File Edit Format View Help
[well_known_ports_kv]
enforceTypes = true
field.port = number
field.transport = string
field.description = string
```

notepad transforms.conf

transforms.conf - Notepad

```
File Edit Format View Help
[well_known_ports]
filename = well_known_ports.csv

[well_known_ports_kv]
external_type = kvstore
collection = well_known_ports_kv
case_sensitive_match = false
fields_list = port, transport, description
```

<http://localhost:8000/en-US/debug/refresh>

```
| makeresults
| eval port="443", transport="tcp", description="https"
| append
  [| makeresults
   | eval port="123", transport="udp", description="ntp"]
| append
  [| makeresults
   | eval port="53", transport="udp", description="ssh"]
| fields - _time
| outputlookup well_known_ports_kv
```

SearchAnalyticsDatasetsReportsAlertsDashboards

Search & Reporting

New Search

Save AsCreate Table ViewClose

1 | makeresults  
2 | eval port="443", transport="tcp", description="https"  
3 | append  
4 | [ makeresults  
5 | | eval port="123", transport="udp", description="ntp"]  
6 | append  
7 | [ makeresults  
8 | | eval port="53", transport="udp", description="ssh"]  
9 | fields - \_time  
10 | outputlookup well\_known\_ports\_kv

Last 24 hours

3 results (3/26/24 6:00:00.000 AM to 3/27/24 6:53:03.000 AM)No Event SamplingJobPauseStopRefreshDownloadVerbose Mode

Events (0)PatternsStatistics (3)Visualization

20 Per PageFormatPreview

| description | port | transport |
|-------------|------|-----------|
| https       | 443  | tcp       |
| ntp         | 123  | udp       |
| ssh         | 53   | udp       |

SearchAnalyticsDatasetsReportsAlertsDashboards

Search & Reporting

New Search

Save AsCreate Table ViewClose

1 | index=botsv2 sourcetype="pan:traffic" earliest=0  
2 | lookup well\_known\_ports\_kv port AS dest\_port, transport  
3 | table src\_ip, dest\_ip, dest\_port, transport, description

All time

85,525 of 85,525 events matchedNo Event SamplingJobPauseStopRefreshDownloadVerbose Mode

Events (85,525)PatternsStatistics (60,524)Visualization

20 Per PageFormatPreview< Prev12345678...Next >

| src_ip     | dest_ip       | dest_port | transport | description |
|------------|---------------|-----------|-----------|-------------|
| 10.0.2.101 | 10.0.1.100    | 53        | udp       | ssh         |
| 10.0.2.107 | 10.0.1.120    | 8014      | tcp       |             |
| 10.0.2.103 | 10.0.1.100    | 445       | tcp       |             |
| 10.0.2.101 | 75.98.70.160  | 80        | tcp       |             |
| 10.0.1.100 | 8.8.8.8       | 53        | udp       | ssh         |
| 10.0.1.200 | 52.40.10.231  | 443       | tcp       | https       |
| 10.0.2.101 | 23.63.227.171 | 80        | tcp       |             |
| 10.0.2.101 | 10.0.1.120    | 8014      | tcp       |             |
| 10.0.2.101 | 23.63.227.171 | 80        | tcp       |             |

21

### 3. External Lookups:

- ⇒ Uses a python script or binary to fetch data from an external source
- ⇒ Example: Built-in dnslookup

In cmd ~ \etc\system\default

notepad transforms.conf

```
transforms.conf - Notepad
File Edit Format View Help
[colon-line]
REGEX = ^(\w+)\s*:[ \t]*(.*)$
FORMAT = $1::$2

[bracket-space]
REGEX = \[(\S+) (.*)\]
FORMAT = $1::$2

[db2]
REGEX = ([A-Z]+) *: (.*)?(?=\n|$| +[A-Z]+ *: )
FORMAT = $1::$2

# Example external lookup
[dnslookup]
python.version = python3
external_cmd = external_lookup.py clienthost clientip
fields_list = clienthost,clientip

[registry]
DELIMS="\n","="
```

We can see exemple of external lookups in pythom scripts in transforms.conf

We can find the external\_looukus.py exist in ~/etc/system/bin

|            |       |    |       |                              |
|------------|-------|----|-------|------------------------------|
| 02/29/2024 | 05:20 | PM | <DIR> | .                            |
| 02/29/2024 | 05:20 | PM | <DIR> | ..                           |
| 02/06/2024 | 10:36 | PM |       | 2,374 addtail.py             |
| 02/06/2024 | 10:36 | PM |       | 3,413 admon.cmd              |
| 02/06/2024 | 10:36 | PM |       | 22,646 appsmanager.py        |
| 02/06/2024 | 10:36 | PM |       | 937 apptemplates.py          |
| 02/06/2024 | 10:36 | PM |       | 2,600 external_lookup.py     |
| 02/06/2024 | 10:36 | PM |       | 8,972 field_extractor.py     |
| 02/06/2024 | 10:36 | PM |       | 5,307 journald.sh            |
| 02/06/2024 | 10:36 | PM |       | 8,483 localapps.py           |
| 02/06/2024 | 10:36 | PM |       | 4,293 logd.sh                |
| 02/06/2024 | 10:36 | PM |       | 912 MonitorNoHandle.cmd      |
| 02/06/2024 | 10:36 | PM |       | 50,475 pdfgen_endpoint.py    |
| 02/06/2024 | 10:36 | PM |       | 4,355 perfmon.cmd            |
| 02/06/2024 | 10:36 | PM |       | 1,654 powershell.cmd         |
| 02/06/2024 | 10:36 | PM |       | 1,656 powershell2.cmd        |
| 02/06/2024 | 10:36 | PM |       | 10,549 sc_rest.py            |
| 02/06/2024 | 10:36 | PM |       | 3,461 secret_tool_keyring.py |
| 02/06/2024 | 10:36 | PM |       | 49 splunk-journald.path      |
| 02/06/2024 | 10:36 | PM |       | 45 splunk-logd.path          |
| 02/06/2024 | 10:36 | PM |       | 16,041 winEventLog.cmd       |
| 02/06/2024 | 10:36 | PM |       | 1,267 WinHostMon.cmd         |
| 02/06/2024 | 10:36 | PM |       | 7,573 WinNetMon.cmd          |

➔ index=botsv2 sourcetype="pan:traffic" earliest=0 site="bea4.cnn.com"  
| lookup dnslookup clienthost AS site

#### 4. Automatic Lookups:

**New Search**

1 index=botsv2 sourcetype="stream:http" earliest=0 site="bea4.cnn.com"  
2 | lookup dnslookup clienthost AS site

✓ 24,743 events (before 3/27/24 7:26:31.000 AM) No Event Sampling

**clientip**

20 Values, 100% of events

Selected ☒ Yes ☐ No

**Reports**

Top values Top values by time Rare values

Events with this field

| Top 10 Values | Count  | %    |
|---------------|--------|------|
| 3.231.143.107 | 24,743 | 100% |
| 3.231.143.108 | 24,743 | 100% |
| 3.231.143.109 | 24,743 | 100% |
| 3.231.143.13  | 24,743 | 100% |
| 3.231.143.15  | 24,743 | 100% |
| 3.231.143.16  | 24,743 | 100% |
| 3.231.143.17  | 24,743 | 100% |
| 3.231.143.19  | 24,743 | 100% |
| 3.231.143.20  | 24,743 | 100% |
| 3.231.143.24  | 24,743 | 100% |

Go to ~\etc\apps\well\_known\_ports\default

notepad props.conf

props.conf - Notepad

File Edit Format View Help

[pan:traffic]  
LOOKUP-ports = well\_known\_ports port AS dest\_port, transport|

refresh by : <http://localhost:8000/en-US/debug/refresh>

**New Search**

1 index=botsv2 sourcetype="pan:traffic" earliest=0  
2 | lookup dnslookup clienthost AS site

123,549 of 123,549 events matched No Event Sampling

**description**

4 Values, 87.796% of events

Selected ☒ Yes ☐ No

**Reports**

Top values Top values by time Rare values

Events with this field

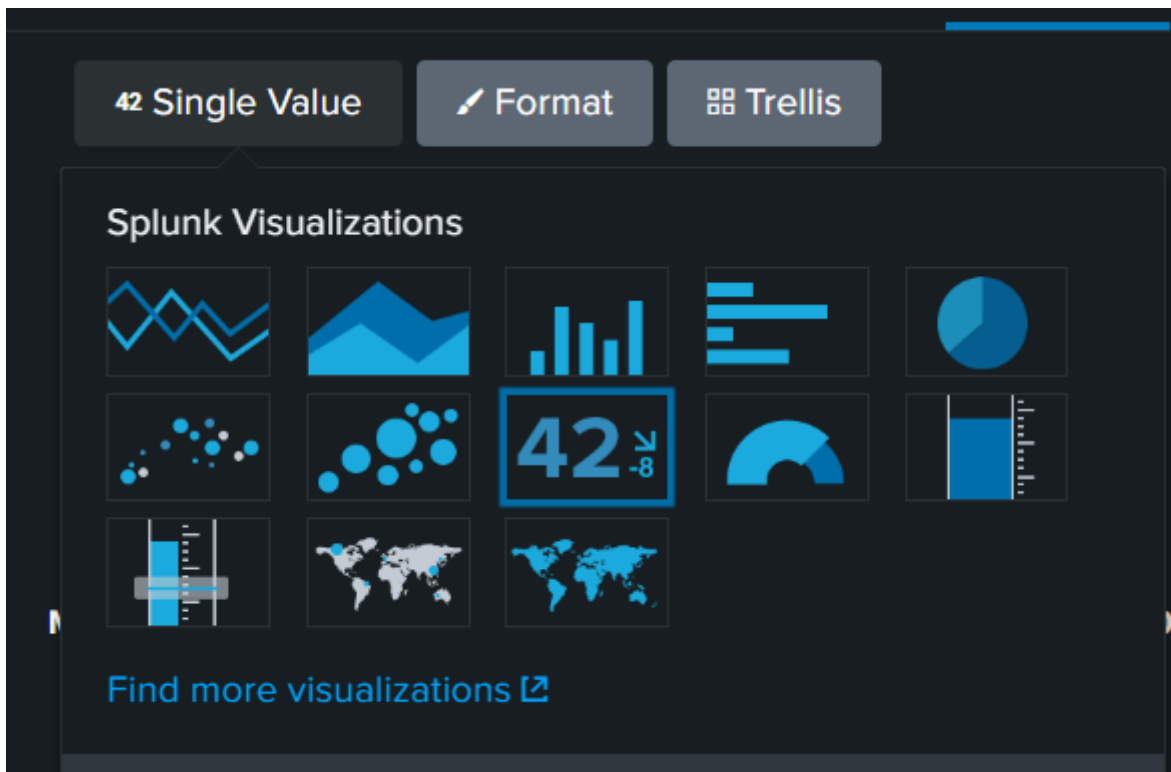
| Values | Count  | %       |
|--------|--------|---------|
| https  | 37,632 | 38.598% |
| dns    | 37,579 | 38.544% |
| http   | 21,552 | 22.105% |
| ntp    | 734    | 0.753%  |

you will find now the field description without use command lookup

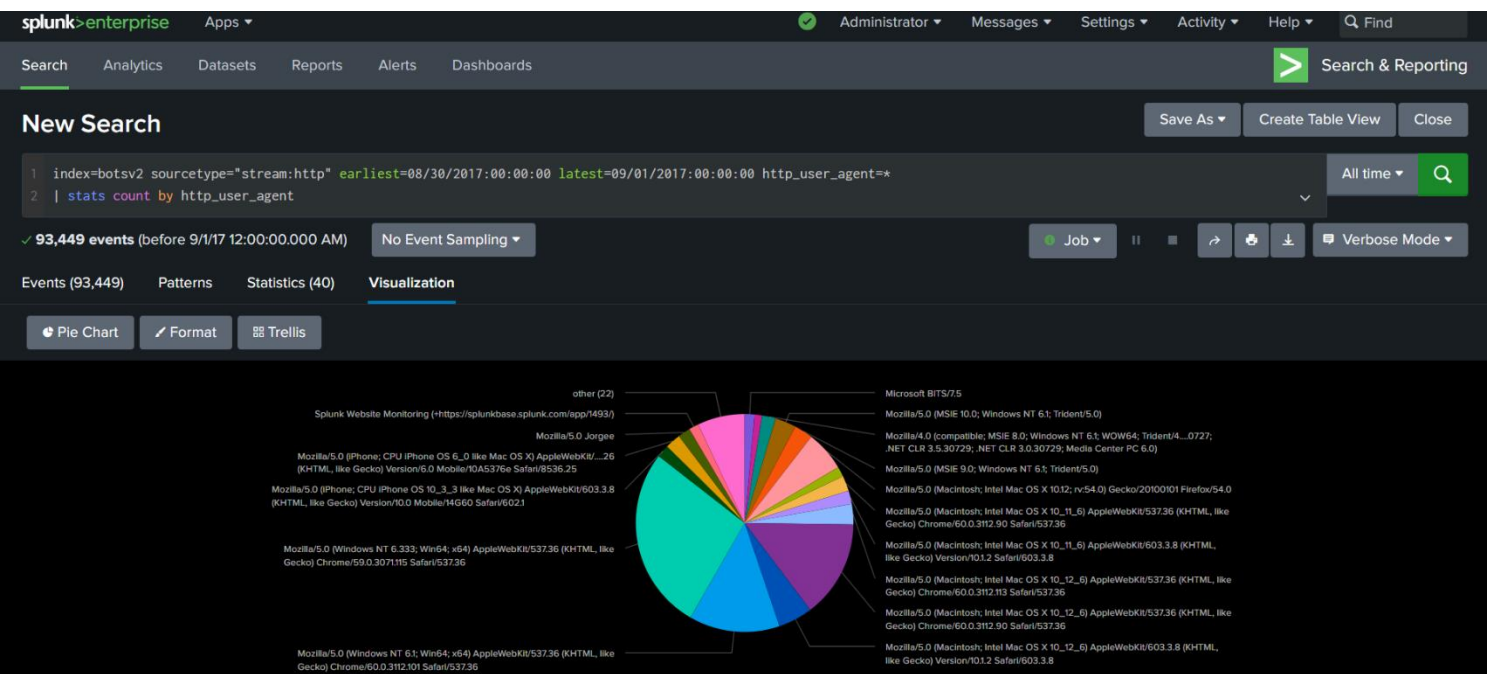


## V. Visualizations and Dashboards:

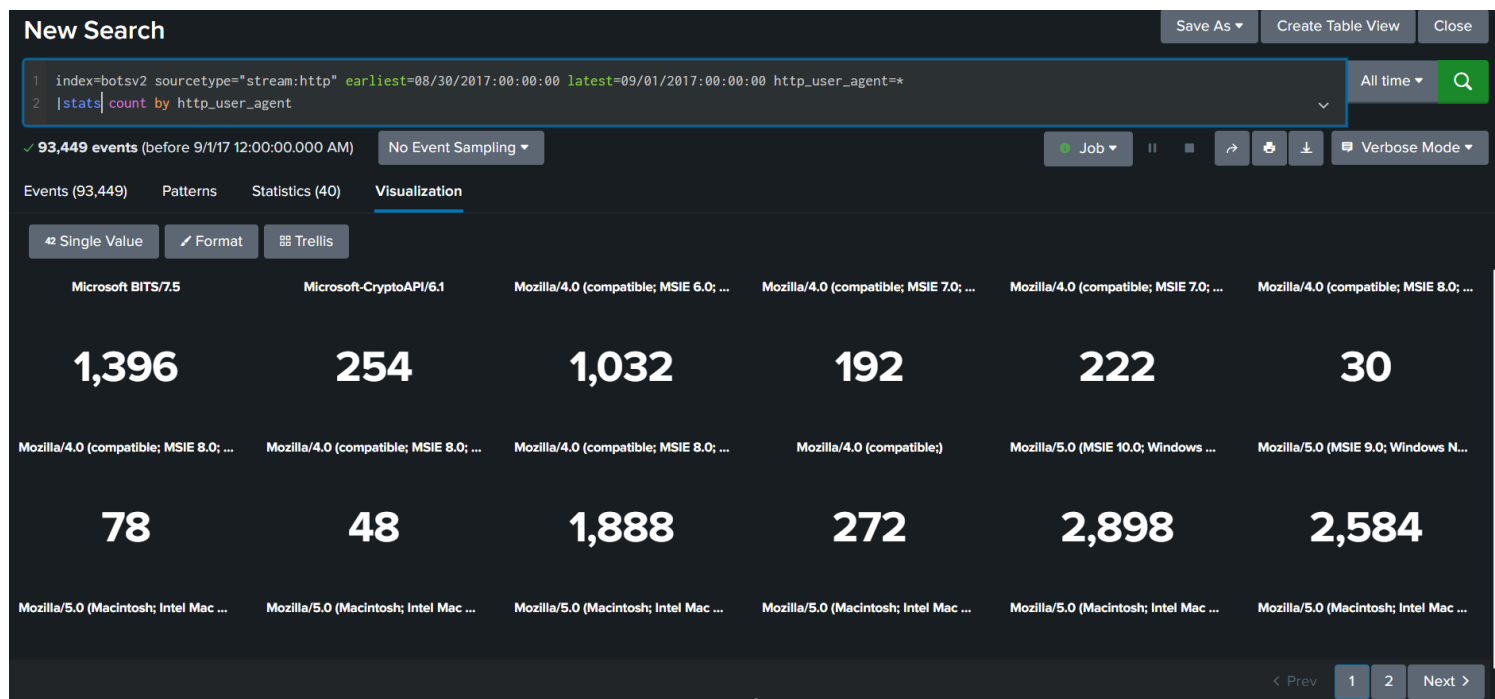
### 1. Visualizations:



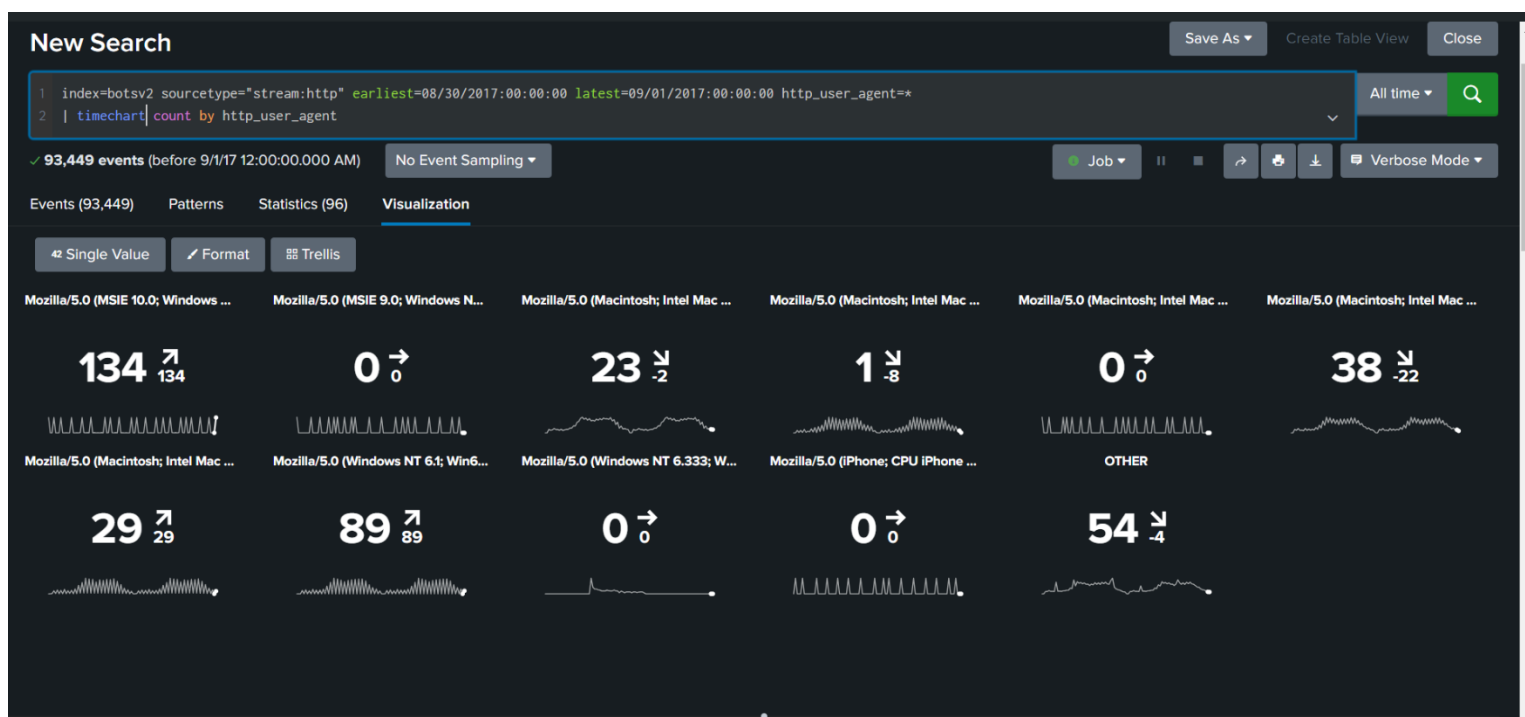
index=botsv2 sourcetype="stream:http" earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00  
http\_user\_agent=\* | stats count by http\_user\_agent



⇒ by using pie chart

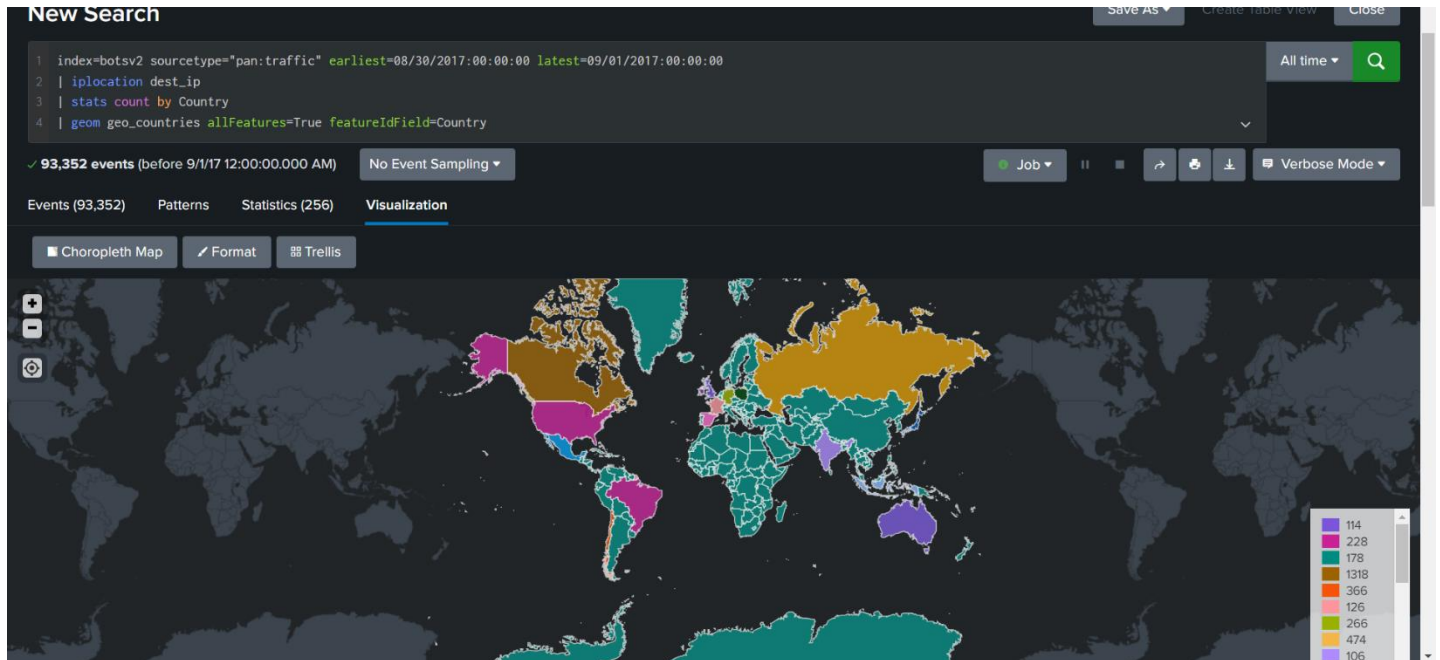


⇒ by using single value with stats command



⇒ By using single value with timechart command

➔ `index=botsv2 sourcetype="pan:traffic" earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00`  
| `iplocation dest_ip`  
| `stats count by Country`  
| `geom geo_countries allFeatures=True featureIdField=Country`



| **iplocation dest\_ip**: This command performs IP geolocation on the destination IP addresses found in the events.

| **stats count by Country**: This command counts the occurrences of events for each country.

| **geom geo\_countries allFeatures=True featureIdField=Country**: This command visualizes the counts on a map, using the country as the feature identifier.

## 2. *Classic Dashboard*

➔ index=botsv2 sourcetype="pan:traffic" earliest=08/30/2017:00:00:00  
latest=09/01/2017:00:00:00  
| stats count BY src\_ip  
| sort -count

The screenshot shows the Splunk Search interface. At the top, the search bar contains the query: `index=botsv2 sourcetype="pan:traffic" earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00 | stats count BY src_ip | sort -count`. Below the search bar, it indicates 93,352 events. The interface is in 'Statistics (15)' view, showing a table of results for 'src\_ip'.

| src_ip     | count |
|------------|-------|
| 10.0.2.101 | 37582 |
| 10.0.1.100 | 13860 |
| 10.0.1.200 | 12840 |
| 10.0.2.107 | 10295 |
| 10.0.2.103 | 4798  |
| 10.0.2.105 | 4322  |

We can create a new dashboard with click in save as new dashboard

The screenshot shows the 'Save Panel to New Dashboard' dialog box. It contains the following fields and options:

- Dashboard Title:** Firewall (with a subtext 'firewall' and an 'Edit ID' link)
- Description:** Optional
- Permissions:** Shared in App (with a dropdown arrow)
- How do you want to build your dashboard?:** Two options are shown: 'Classic Dashboards' (The traditional Splunk dashboard builder) and 'Dashboard Studio' (A new builder to create visually-rich, customizable dashboards, marked as 'NEW').
- Panel Title:** Optional
- Buttons:** Cancel and Save to Dashboard (highlighted in green).

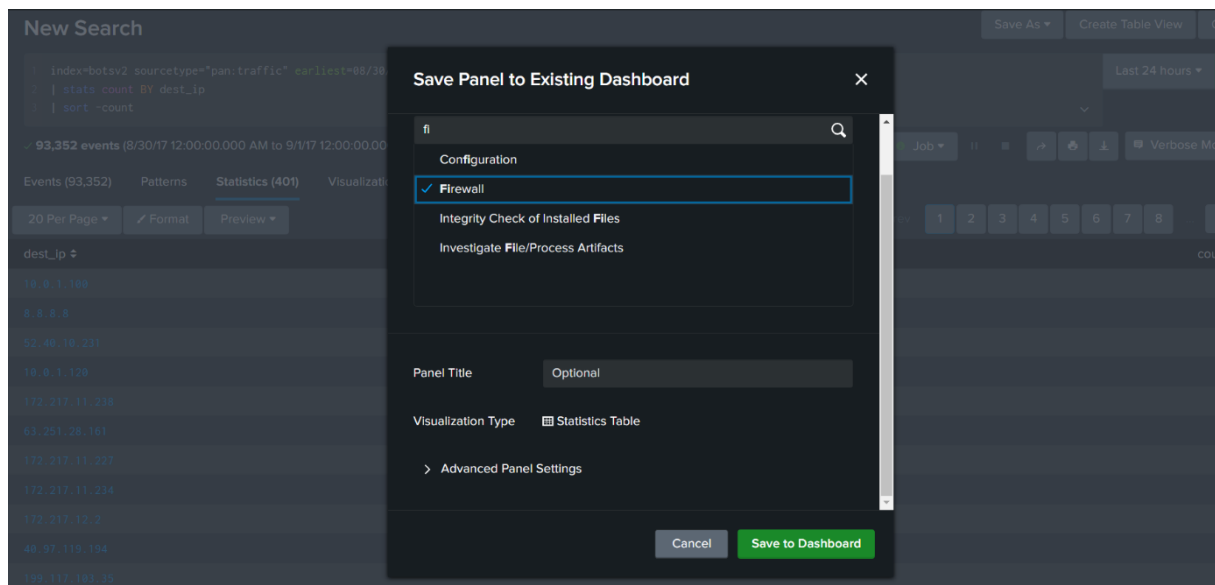
We write a new name for it and we can change its permissions if we want ,Then we can save it

The screenshot shows a Splunk Enterprise dashboard titled "Firewall". It displays a table with two columns: "src\_ip" and "count". The table lists various IP addresses and their corresponding counts. The top of the dashboard includes navigation links like Search, Analytics, Datasets, Reports, Alerts, and Dashboards. There are also buttons for Edit, Export, and a menu icon.

| src_ip     | count |
|------------|-------|
| 10.0.2.101 | 37582 |
| 10.0.1.100 | 13860 |
| 10.0.1.200 | 12840 |
| 10.0.2.107 | 10296 |
| 10.0.2.103 | 4798  |
| 10.0.2.105 | 4322  |
| 10.0.2.110 | 3886  |
| 10.0.2.109 | 3476  |
| 10.0.2.108 | 1276  |
| 10.0.4.4   | 346   |
| 10.0.1.222 | 334   |
| 10.0.2.102 | 138   |
| 10.0.1.220 | 96    |
| 10.0.1.101 | 76    |
| 10.0.4.5   | 26    |

we will get the dashboard above with table of src\_ip

also we can add a more content from another searches into this dashboard



such as statistic of destinations ip by count but before that we will use save as existing dashboard

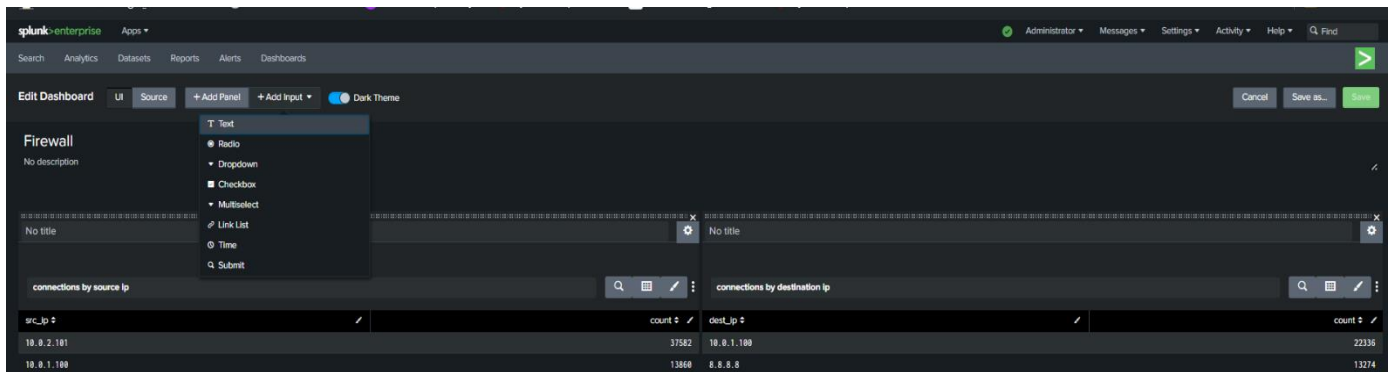
| Firewall   |       | Edit | Export | ... |
|------------|-------|------|--------|-----|
| src_ip     | count |      |        |     |
| 10.0.2.101 | 37582 |      |        |     |
| 10.0.1.100 | 13860 |      |        |     |
| 10.0.1.200 | 12840 |      |        |     |
| 10.0.2.107 | 10296 |      |        |     |
| 10.0.2.103 | 4798  |      |        |     |
| 10.0.2.105 | 4322  |      |        |     |
| 10.0.2.110 | 3886  |      |        |     |
| 10.0.2.109 | 3476  |      |        |     |
| 10.0.2.108 | 1276  |      |        |     |
| 10.0.4.4   | 346   |      |        |     |
| 10.0.1.222 | 334   |      |        |     |
| 10.0.2.102 | 138   |      |        |     |
| 10.0.1.220 | 96    |      |        |     |
| 10.0.1.101 | 76    |      |        |     |
| 10.0.4.5   | 26    |      |        |     |
| dest_ip    | count |      |        |     |
| 10.0.1.100 | 22336 |      |        |     |
| 8.8.8.8    | 13274 |      |        |     |

we will find the second table below the first table, but will do some edits to obtain the dashboard more former. Also, we will add another contents, the first the timechat by dest port, and table of statistic by source and destination interface

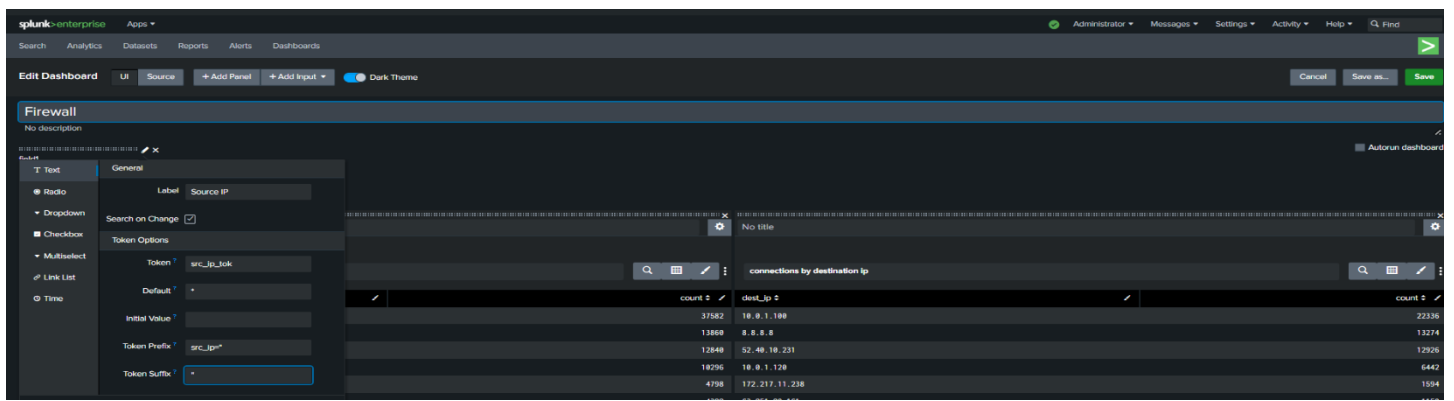


### 3. *building a dynamic Dashboard by token*

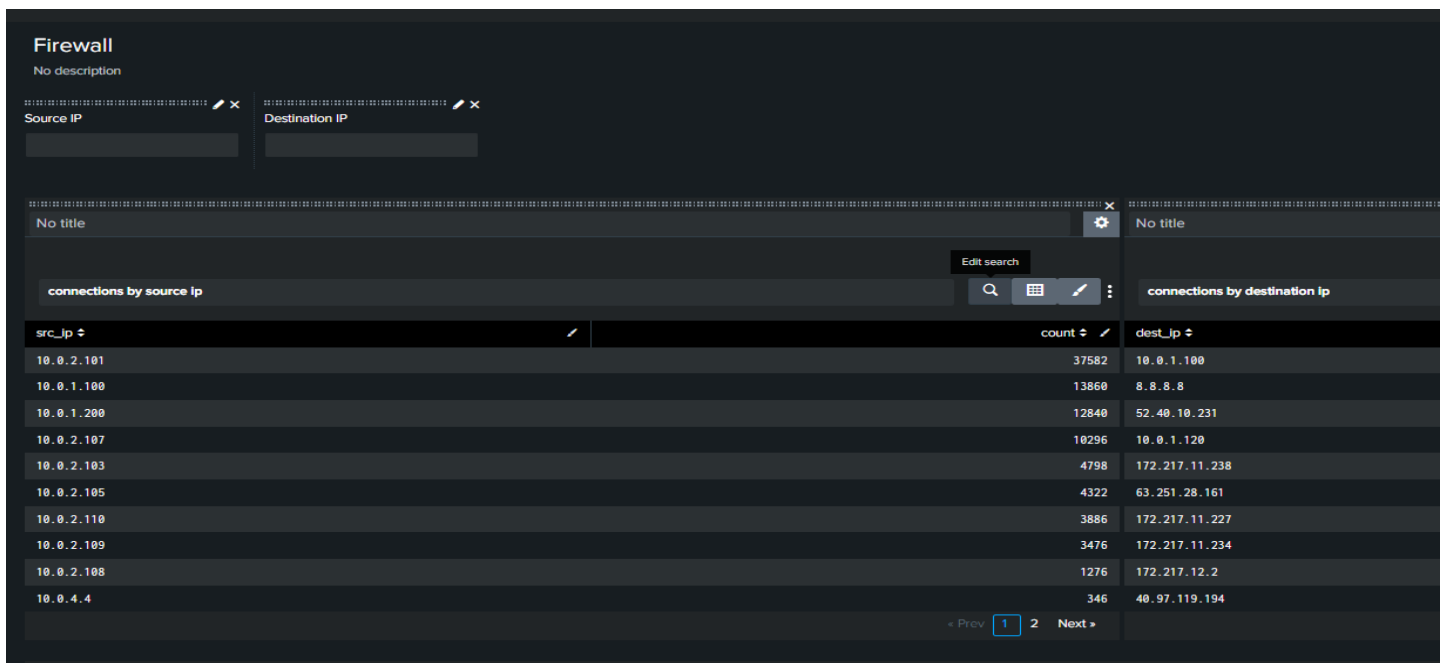
by using the dashboard we was creating before , we click on edit and add input “text”



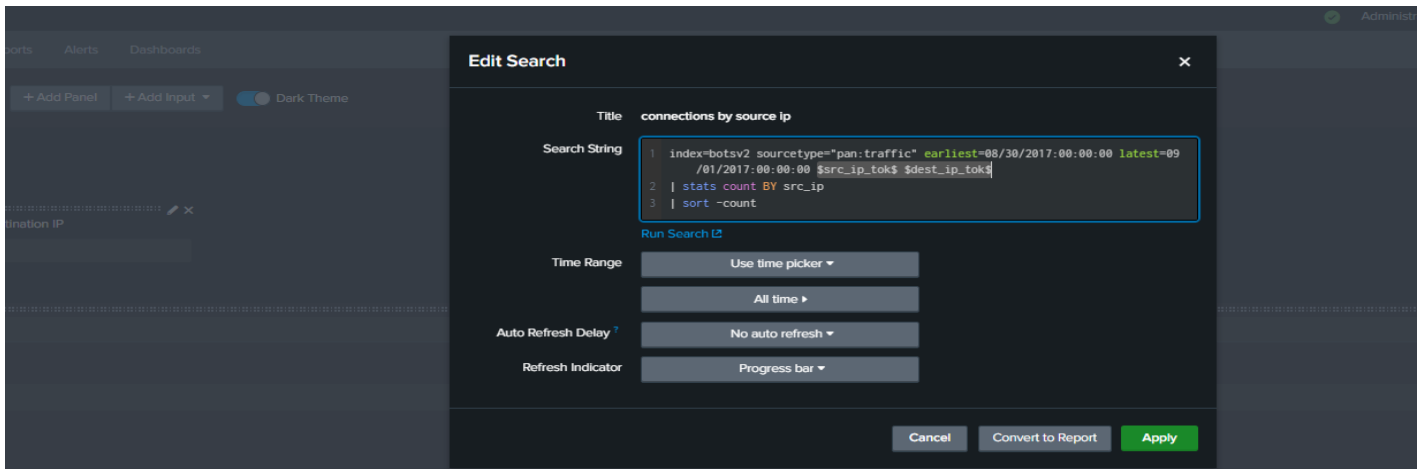
After that, we click on the pen icon and input the information as shown in the image below



then we wil do the same in destination ip :



we will click in search icon from each the four panel and add \$src\_ip\_tok\$ \$dest\_ip\_tok\$ and click on Apply , then we will save all edits



finally, we can write which IP we want to show stastics of her in Dashboard (we can use destantion ip ,source ip or the both of then in the same time ) , the exemple below we show all stastic of conection from all IPs to 8.8.8.8

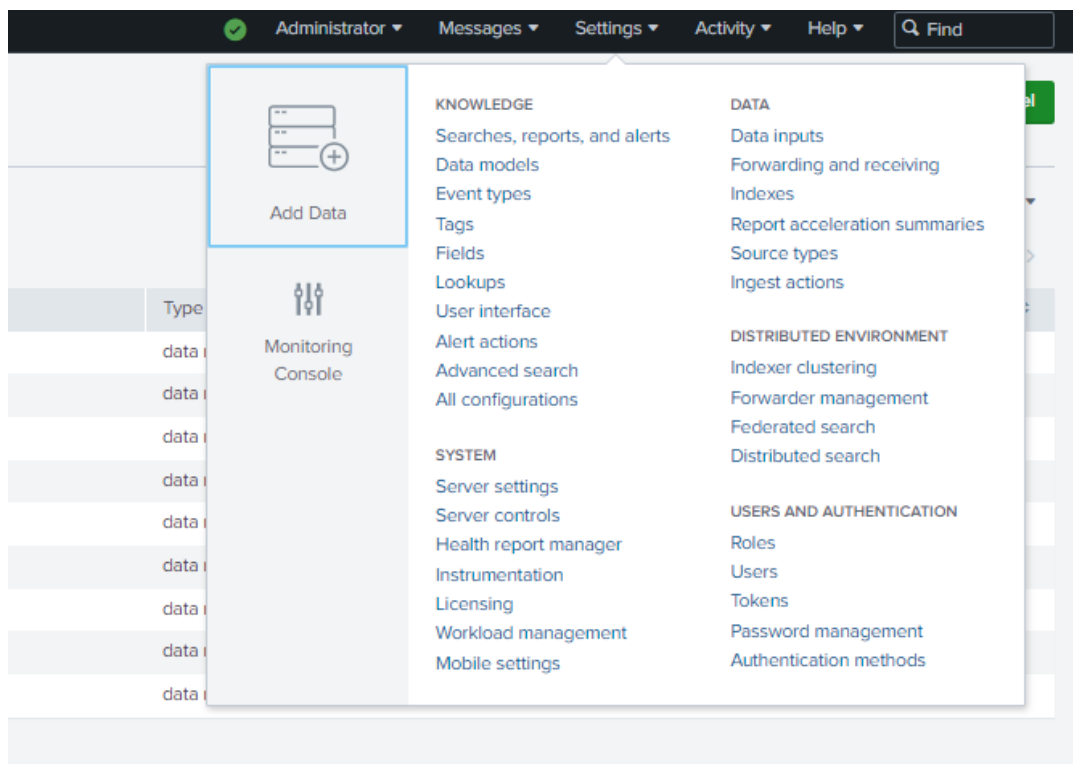




## VI. Data models:

- ⇒ A data model is a collection of important fields for a certain topic:  
Network traffic will always have a source and a destination address
- ⇒ Data models have knowledge about the data built-in:  
Some authentication events are created by logins with privileged accounts
- ⇒ Data models are hierarchical (datasets):  
A subset of all web traffic events may come from proxy servers
- ⇒ Data models are like looking at your data if it was a relational database (columns and rows)

We can use gui by click on settings then data models



we will find a lot of data models we can chose for exmple the network traffic from the second page

The screenshot shows the 'Data Models' page in Splunk. It displays a list of 29 data models. The table has columns: Title, Type, Actions, App, Owner, and Sharing. The first few rows are: 'Network Traffic' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global), 'Performance' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global), 'Splunk Audit Logs' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global), 'Splunk's Internal Audit Logs - SAMPLE' (data model, Edit, Pivot, search, nobody, App), 'Splunk's Internal Server Logs - SAMPLE' (data model, Edit, Pivot, search, nobody, App), 'Ticket Management' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global), 'Updates' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global), 'Vulnerabilities' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global), and 'Web' (data model, Edit, Pivot, Splunk\_SA\_CIM, nobody, Global). The 'Pivot' button is highlighted for the 'Network Traffic' model.

after that , we click on pivot then all traffic

**Select a Dataset**

4 Objects in Network Traffic

- All Traffic
- Traffic By Action
- Allowed Traffic
- Blocked Traffic

61,690 of 61,690 events matched

Filters: All time

Split Rows: +

Count of All Traffic: 61,690

Split Columns: +

Column Values: Count of All T...

we can choose fields now , for exmple destination ip :

**New Pivot**

Filters: All time

Field: a action, a app, # bytes, # bytes\_in, # bytes\_out, # channel, a dest, a dest\_built, a dest\_category, a dest\_interface, a dest\_ip, a dest\_mac, # dest\_port, a dest\_priority, a dest\_translated\_ip

Split Columns: +

Column Values: Count of All T...

**New Pivot**

78,960 of 78,960 events matched

Filters: All time

Split Rows: dest\_ip

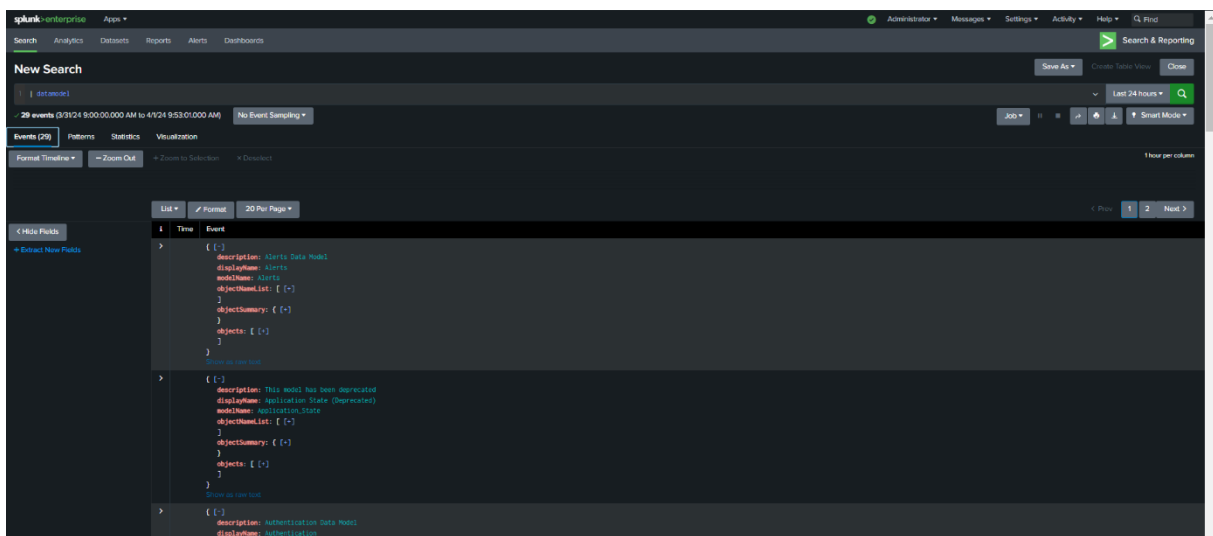
Count of All Traffic: 78,960

| dest_ip      | Count of All Traffic |
|--------------|----------------------|
| 4.4.4.4      | 71                   |
| 5.128.19.197 | 11                   |
| 8.8.8.8      | 8988                 |
| 8.18.45.31   | 58                   |
| 8.43.72.21   | 25                   |
| 8.43.72.24   | 48                   |
| 8.43.72.27   | 11                   |
| 8.43.72.32   | 22                   |
| 8.43.72.42   | 228                  |
| 8.43.72.44   | 48                   |
| 8.43.72.52   | 13                   |
| 8.43.72.58   | 14                   |
| 8.43.72.112  | 12                   |
| 10.8.1.188   | 2892                 |
| 10.8.1.181   | 159                  |
| 10.8.1.128   | 1493                 |
| 10.8.1.208   | 76                   |
| 10.8.1.228   | 11                   |
| 10.8.1.205   | 152                  |
| 10.8.2.181   | 12                   |

we can visualite data , thiss all without using SPL query

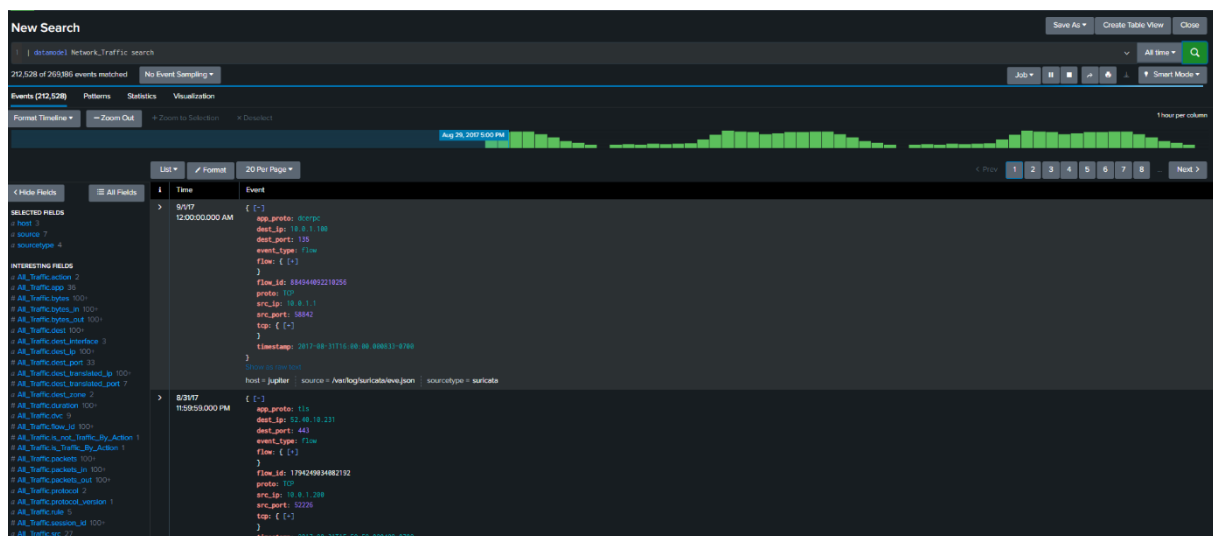


we can also use data models in bar of search



we can select which data model we want by click in or by using saerch

➡ | datamodel Network\_Traffic search



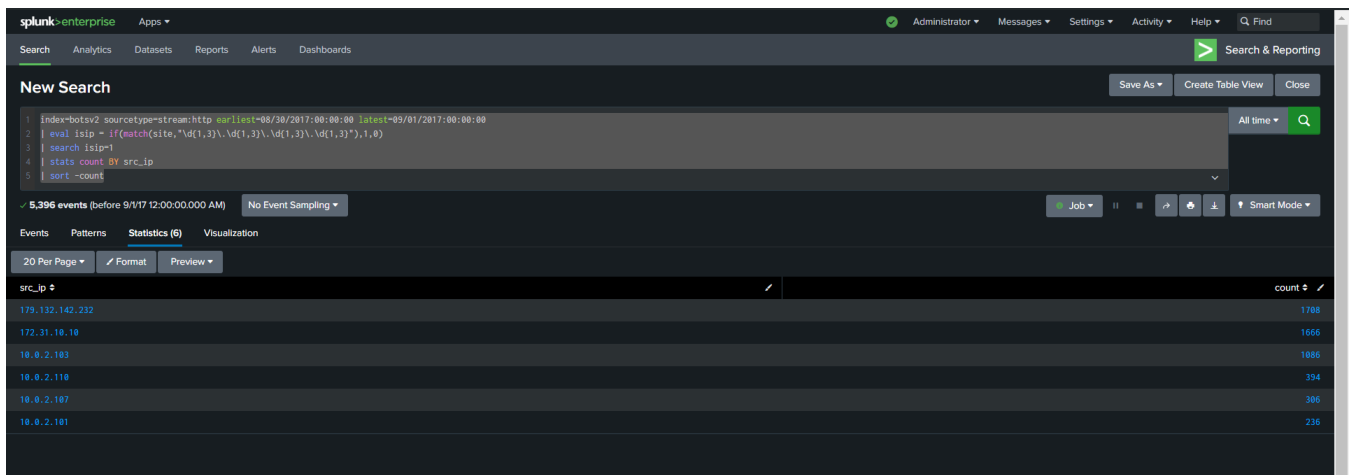
[illegible]

## VII. Using Search to Find Suspicious Events:

### 1. HTTP Connections to a Server by IP (Conditional Eval)

*Searching for connections using the IP directly:*

```
➔ index=botsv2 sourcetype=stream:http earliest=08/30/2017:00:00:00
latest=09/01/2017:00:00:00
| eval isip = if(match(site,"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"),1,0)
| search isip=1
| stats count BY src_ip
| sort -count
```



The screenshot shows the Splunk Enterprise Search interface. The search bar contains the following query:

```
1 index=botsv2 sourcetype=stream:http earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00
2 | eval isip = if(match(site,"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"),1,0)
3 | search isip=1
4 | stats count BY src_ip
5 | sort -count
```

The search results are displayed in a table with the following columns: **src\_ip** and **count**. The results are sorted by count in descending order.

| src_ip          | count |
|-----------------|-------|
| 179.132.142.232 | 1788  |
| 172.31.18.18    | 1666  |
| 18.8.2.183      | 1886  |
| 18.8.2.118      | 394   |
| 18.8.2.187      | 306   |
| 18.8.2.181      | 238   |

| **eval isip = if(match(site,"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"),1,0)**: This evaluates whether the "site" field contains an IP address using a regular expression. If an IP address is found, it assigns 1 to the field "isip"; otherwise, it assigns 0.

| **search isip=1**: This filters out events where "isip" is equal to 1, meaning that the "site" field contains an IP address.

## 2. Searching 2 Data Sources for a Common Indicator (Using OR)

Using OR to combine the datasets (Searching in stream:http and stream:dns):

```
➔ index=botsv2 (sourcetype=stream:http site="uranus.frothly.local:8014") OR  
(sourcetype=stream:dns query={"uranus.frothly.local"}) earliest=08/20/2017:00:00:00  
latest=09/01/2017:00:00:00  
| eval site = replace(site,"(.*)\d+","\1")  
| eval dest_host = if(sourcetype=="stream:http",site,'query{')  
| search dest_host="uranus.frothly.local"  
| stats count values(sourcetype) AS sourcetype BY src_ip, dest_host  
| sort -count
```

The screenshot shows the Splunk Enterprise search interface. The search bar contains the following query:

```
index=botsv2 (sourcetype=stream:http site="uranus.frothly.local:8014") OR (sourcetype=stream:dns query={"uranus.frothly.local"}) earliest=08/20/2017:00:00:00 latest=09/01/2017:00:00:00  
| eval site = replace(site,"(.*)\d+","\1")  
| eval dest_host = if(sourcetype=="stream:http",site,'query{')  
| search dest_host="uranus.frothly.local"  
| stats count values(sourcetype) AS sourcetype BY src_ip, dest_host  
| sort -count
```

The results table shows 8,305 events. The table has columns for src\_ip, dest\_host, count, and sourcetype. The results are sorted by count in descending order.

| src_ip     | dest_host            | count | sourcetype                |
|------------|----------------------|-------|---------------------------|
| 10.0.2.107 | uranus.frothly.local | 5074  | stream:dns<br>stream:http |
| 10.0.2.110 | uranus.frothly.local | 2992  | stream:dns<br>stream:http |
| 10.0.2.105 | uranus.frothly.local | 833   | stream:dns<br>stream:http |
| 10.0.2.103 | uranus.frothly.local | 598   | stream:dns<br>stream:http |
| 10.0.2.108 | uranus.frothly.local | 384   | stream:dns                |
| 10.0.2.101 | uranus.frothly.local | 322   | stream:dns<br>stream:http |
| 10.0.1.128 | uranus.frothly.local | 4     | stream:dns                |

**eval site = replace(site,"(.\*)\d+","\1"):** This uses the eval command to create a new field "site" by removing the port number from the "site" field.

- **replace(site,"(.\*)\d+","\1"):** This is the function used to modify the "site" field. It's a regular expression-based replacement function.
- **(.\*)\d+:** This is a regular expression pattern. Let's break it down:
  - **.\***: This matches any character (except for line terminators) zero or more times. Essentially, it matches any sequence of characters.
  - **::** This matches the colon character.
  - **\d+:** This matches one or more digits.
- **\1:** This is a backreference in the replacement string. It refers to the first captured group **(.\*)**, which is everything before the colon. So essentially, it replaces the entire match

**eval dest\_host = if(sourcetype=="stream:http",site,'query{'):** This creates a new field "dest\_host". If the sourcetype is "stream:http", it uses the "site" field; otherwise, it uses the "query{" field.

Overall, this query is designed to search for events related to the "uranus.frothly.local" host within the specified time range and analyze the counts and types of those events based on the source IP address.

### 3. Finding Traces of Look-a-like Domains (Using fuzzylookup)

#### Calculating the Levenshtein Distanc:

```
➔ index=botsv2 (sourcetype=stream:smtp sender_domain=*) OR (sourcetype=stream:dns
query{}=*) OR (sourcetype=stream:http site=*) earliest=1
| eval maybe_evil_domain = coalesce(sender_domain,'query{','site)
| stats count BY maybe_evil_domain, sourcetype
| fuzzylookup addmetrics=true our_domains domains AS maybe_evil_domain
| search fuzzy_score < 3
| table domains maybe_evil_domain fuzzy_score fuzzy_similarity sourcetype
```

The screenshot shows the 'Browse More Apps' section in Splunk. A search bar contains 'fuzzylookup'. On the left, a list of categories is shown with checkboxes. The 'Fuzzylookup' app is displayed as the 'Best Match'. It includes an 'Install' button, a description of its functionality (applying fuzzy logic to lookups), and a list of applications (Domain analysis, Blacklist similarity, Typo identification). Metadata at the bottom indicates it is in the 'Security, Fraud & Compliance, IT Operations' category, authored by 'Deductiv Inc', with 844 downloads and released 2 years ago.

First, we need to instal Fuzzylookup in splunk enterprise , then we will create file lookups her name is our\_domain (we flow the same steps of creation CSV file lookups)

The screenshot shows a Notepad window titled 'our\_domain.csv'. The menu bar includes 'File', 'Edit', 'Format', 'View', and 'Help'. The text content of the file is as follows:

```
domains
broth.ly
brothly.local
www.broth.ly
brothly.com
```

The screenshot shows the Splunk Search interface. The search bar contains 'inputlookup our\_domains'. The results show 4 items: 'broth.ly', 'brothly.local', 'www.broth.ly', and 'brothly.com'. The interface includes navigation tabs for 'Search', 'Analytics', 'Datasets', 'Reports', 'Alerts', and 'Dashboards'. The search results are displayed in a table format with columns for 'domains'.

The screenshot shows the Splunk Enterprise interface with a new search query entered in the search bar. The query is:

```
1 index=botsv2 (sourcetype=stream:smtp sender_domain=*) OR (sourcetype=stream:dns query=*) OR (sourcetype=stream:http site=*) earliest=1
2 | eval maybe_evil_domain = coalesce(sender_domain,'query','site')
3 | stats count BY maybe_evil_domain, sourcetype
4 | fuzzylookup addmetrics=true our_domains domains AS maybe_evil_domain
5 | search fuzzy_score < 3
6 | table domains maybe_evil_domain fuzzy_score fuzzy_similarity sourcetype
```

The search results are displayed in a table format with the following columns: domains, maybe\_evil\_domain, fuzzy\_score, fuzzy\_similarity, and sourcetype. The results show a list of domains and their corresponding similarity scores and source types.

| domains       | maybe_evil_domain | fuzzy_score | fuzzy_similarity | sourcetype  |
|---------------|-------------------|-------------|------------------|-------------|
| brothly.local | FROTHLY.LOCAL     | 1           | 92.31            | stream:dns  |
| broth.ly      | froth.ly          | 1           | 87.5             | stream:dns  |
| broth.ly      | froth.ly          | 1           | 87.5             | stream:smtp |
| brothly.local | frothly.local     | 1           | 92.31            | stream:dns  |
| www.broth.ly  | www.froth.ly      | 1           | 91.67            | stream:dns  |
| www.broth.ly  | www.froth.ly      | 1           | 91.67            | stream:http |

**fuzzylookup addmetrics=true our\_domains domains AS maybe\_evil\_domain:** This command performs a fuzzy lookup against a list of known "safe" domains. Here's what each part does:

- **fuzzylookup:** This command performs a fuzzy lookup operation.
- **addmetrics=true:** This parameter specifies that additional metrics should be added to the results.
- **our\_domains:** This likely represents a list of known-safe domains maintained by the organization.
- **domains AS maybe\_evil\_domain:** This specifies the field from the fuzzy lookup results to be compared against the **maybe\_evil\_domain** field from the original data.

**search fuzzy\_score < 3:** This line filters the results to only include matches where the fuzzy score (a measure of similarity between strings) is less than 3. This implies that the domains being compared are quite different from the known-good domains.

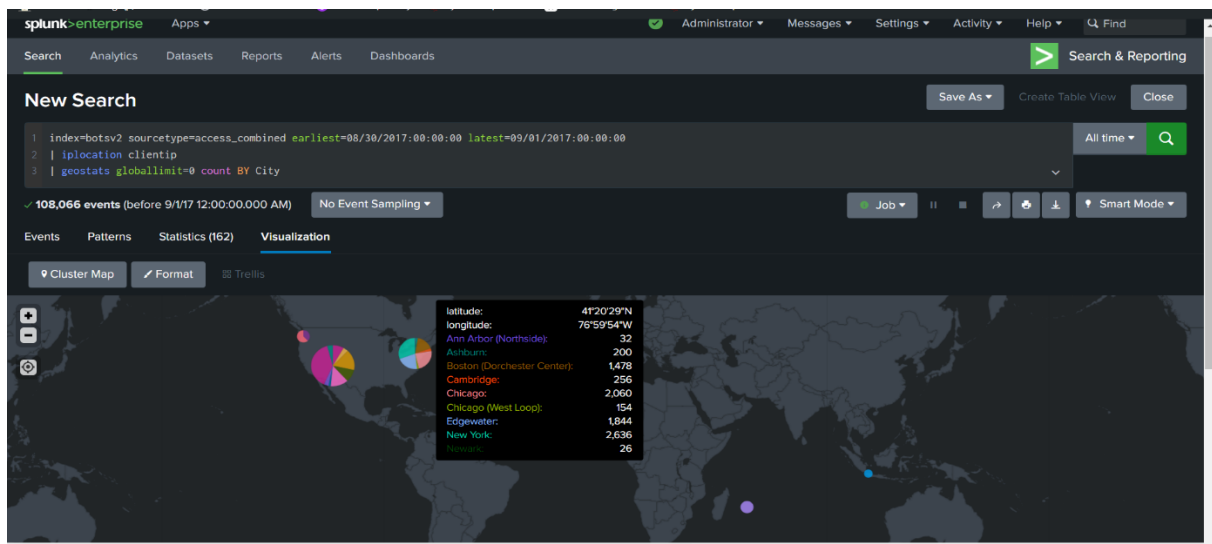
this query is designed to identify potentially malicious domains by comparing them against a list of known-safe domains. It calculates similarity scores between potentially malicious domains and known-safe domains using fuzzy matching and filters the results to only show domains with low similarity scores. The results are then presented in a table format for further analysis.



#### 4. Using Geolocation to Find Anomalous Connections

##### Finding the Rarest Countries in Connection Events:

➔ `index=botsv2 sourcetype=access_combined earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00`  
`| iplocation clientip | geostats globallimit=0 count BY City`



**iplocation clientip:** This command is used to perform IP geolocation. It extracts location information (such as city, country, etc.) from the IP addresses found in the "clientip" field.

**geostats globallimit=0 count BY City:** This command calculates geographical statistics based on the location information extracted in the previous step. Specifically, it counts the occurrences of events grouped by the "City" field. The **globallimit=0** parameter ensures that all results are returned, regardless of any default limits set in the Splunk configuration.

➔ `index=botsv2 sourcetype=access_combined earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00`  
`| iplocation clientip`  
`| rare Country | search percent < 1`

The screenshot shows the Splunk Enterprise interface. The search bar contains the following query: `1 index=botsv2 sourcetype=access_combined earliest=08/30/2017:00:00:00 latest=09/01/2017:00:00:00`  
`2 | iplocation clientip`  
`3 | rare Country`  
`4 | search percent < 1`  
The results show 108,066 events. The statistics tab is selected, displaying a table of rare countries.

| Country   | count | percent  |
|-----------|-------|----------|
| Germany   | 30    | 0.122841 |
| Indonesia | 34    | 0.138313 |

1. **rare Country:** This command identifies rare values in the "Country" field. It counts the occurrences of each unique country and calculates the percentage of occurrences relative to the total. Countries with a percentage of occurrences less than 1% are considered rare.
2. **search percent < 1:** This filters the results to only include countries with occurrence percentages less than 1%. These are the rare countries identified in the access logs within the specified time range.

## 5. first Time Login of a User on a Machine

### First Time Logins of User Accounts:

```
➔ index=botsv2 sourcetype=wineventlog OR sourcetype=xmlwineventlog signature_id=4624
NOT user="ANONYMOUS LOGON" earliest=08/01/2017:00:00:00 latest=08/25/2017:00:00:00
| stats earliest(_time) AS earliest latest(_time) AS latest BY user, dest
| eval deadline = strptime("08/25/2017:00:00:00", "%m/%d/%Y:%H:%M:%S")
| where earliest >= relative_time(deadline, "-1d@d")
| convert ctime(earliest) AS earliest
| table user, dest, earliest
```

The screenshot shows the Splunk Search interface. At the top, a 'New Search' bar contains the query: `index=botsv2 sourcetype=wineventlog OR sourcetype=xmlwineventlog signature_id=4624 NOT user="ANONYMOUS LOGON" earliest=08/01/2017:00:00:00 latest=08/25/2017:00:00:00 | stats earliest(_time) AS earliest latest(_time) AS latest BY user, dest | eval deadline = strptime("08/25/2017:00:00:00", "%m/%d/%Y:%H:%M:%S") | where earliest >= relative_time(deadline, "-1d@d") | convert ctime(earliest) AS earliest | table user, dest, earliest`. Below the search bar, it indicates '307,248 events (before 8/25/17 12:00:00.000 AM)' and 'No Event Sampling'. The results are displayed in a table with columns: user, dest, and earliest. The table shows two distinct users, 'billy.tun' and 'service3', each with multiple login events to different destinations.

| user      | dest                      | earliest            |
|-----------|---------------------------|---------------------|
| billy.tun | venus.frothly.local       | 08/24/2017 04:50:57 |
| billy.tun | wrk-abungst.frothly.local | 08/24/2017 04:50:57 |
| billy.tun | wrk-aturing.frothly.local | 08/24/2017 04:50:57 |
| billy.tun | wrk-bgist.frothly.local   | 08/24/2017 04:50:57 |
| billy.tun | wrk-fmaltes.frothly.local | 08/24/2017 04:50:56 |
| billy.tun | wrk-klagerf.frothly.local | 08/24/2017 04:50:57 |
| service3  | venus.frothly.local       | 08/24/2017 04:55:14 |
| service3  | wrk-abungst.frothly.local | 08/24/2017 04:55:13 |

**stats earliest(\_time) AS earliest latest(\_time) AS latest BY user, dest:** Groups the events by the "user" and "dest" fields and calculates the earliest and latest timestamps for each group

**eval deadline = strptime("08/25/2017:00:00:00", "%m/%d/%Y:%H:%M:%S"):** Converts the deadline date string "08/25/2017:00:00:00" into an epoch time format using the **strptime** function.

**where earliest >= relative\_time(deadline, "-1d@d"):** Filters the results to only include events where the earliest timestamp is greater than or equal to one day before the deadline specified (i.e., August 24, 2017).

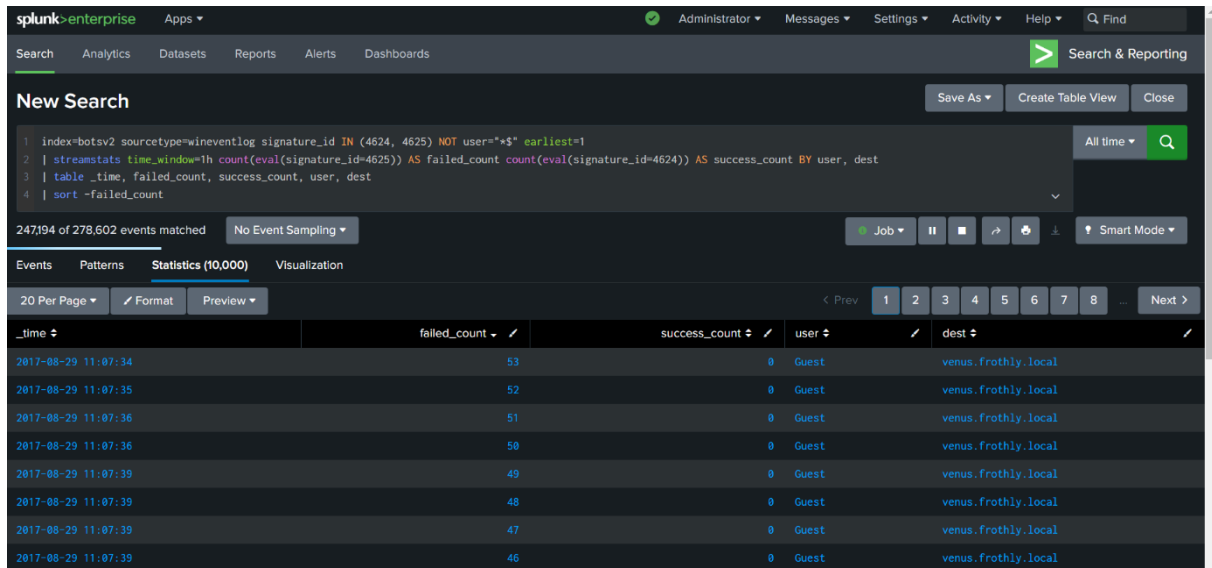
**convert ctime(earliest) AS earliest:** Converts the earliest timestamp back to human-readable format using the **convert** and **ctime** functions.

By using this query we see in the above image two users logged in different destination, so this is look a suspicious events

## 6. Identifying Password Guessing Attempts with Failed and Successful Logins

### Failed and Successful Login Attempts:

➔ `index=botsv2 sourcetype=wineventlog signature_id IN (4624, 4625) NOT user="*$" earliest=1`  
`| streamstats time_window=1h count(eval(signature_id=4625)) AS failed_count`  
`count(eval(signature_id=4624)) AS success_count BY user, dest`  
`| table _time, failed_count, success_count, user, dest`



The screenshot shows the Splunk Enterprise interface with a search results table. The search bar contains the query: `index=botsv2 sourcetype=wineventlog signature_id IN (4624, 4625) NOT user="*$" earliest=1`. The results table has columns: `_time`, `failed_count`, `success_count`, `user`, and `dest`. The data shows failed login attempts for the user 'Guest' to the destination 'venus.frothly.local'.

| _time               | failed_count | success_count | user  | dest                |
|---------------------|--------------|---------------|-------|---------------------|
| 2017-08-29 11:07:34 | 53           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:35 | 52           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:36 | 51           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:36 | 50           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:39 | 49           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:39 | 48           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:39 | 47           | 0             | Guest | venus.frothly.local |
| 2017-08-29 11:07:39 | 46           | 0             | Guest | venus.frothly.local |

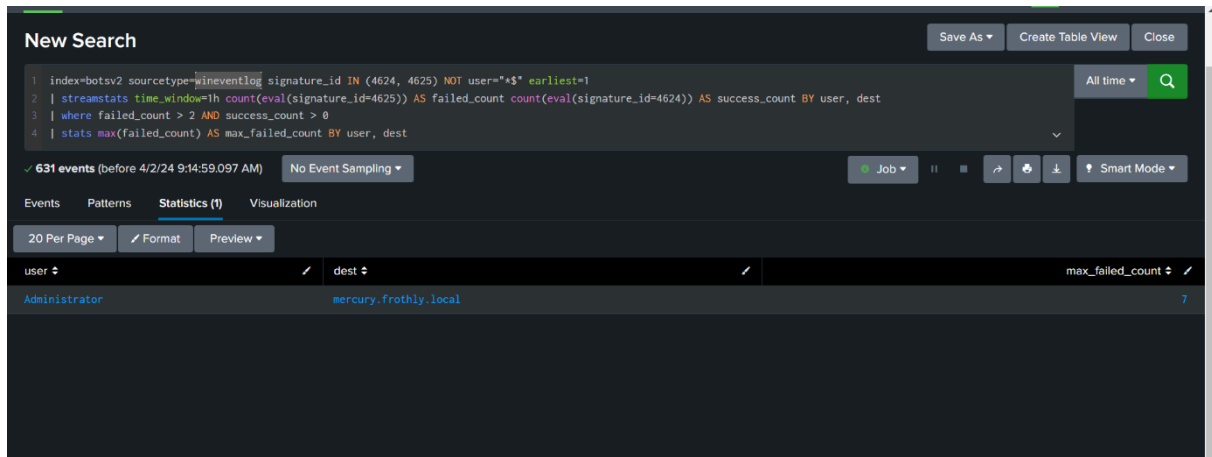
**signature\_id IN (4624, 4625):** Filters events to include those with either signature ID 4624 or 4625. Signature ID 4624 typically corresponds to successful logon events, while 4625 corresponds to failed logon events.

**NOT user="\*\$":** Excludes logon events where the user ends with a "\$". These events typically represent computer account logons, which might not be relevant for user-based analysis.

**streamstats time\_window=1h count(eval(signature\_id=4625)) AS failed\_count count(eval(signature\_id=4624)) AS success\_count BY user, dest:** This command calculates statistics over a streaming window of 1 hour. It counts the occurrences of signature IDs 4625 (failed logon attempts) and 4624 (successful logon attempts) for each user and destination combination.

this query aims to provide insights into user logon activities by counting the number of successful and failed logon attempts over time, grouped by user and destination. However, it's worth noting that the **earliest=1** parameter seems incomplete and may need to be adjusted to specify a valid time range for the search.

➔ index=botsv2 sourcetype=wineventlog signature\_id IN (4624, 4625) NOT user="\*\$" earliest=1  
 | streamstats time\_window=1h count(eval(signature\_id=4625)) AS failed\_count  
 count(eval(signature\_id=4624)) AS success\_count BY user, dest  
 | where failed\_count > 2 AND success\_count > 0  
 | stats max(failed\_count) AS max\_failed\_count BY user, dest



**where failed\_count > 2 AND success\_count > 0:** Filters the results to only include combinations where there are more than 2 failed logon attempts (**failed\_count > 2**) and at least 1 successful logon attempt (**success\_count > 0**).

**stats max(failed\_count) AS max\_failed\_count BY user, dest:** Aggregates the results to show the maximum number of failed logon attempts (**max\_failed\_count**) for each user and destination combination

this query aims to identify instances where there have been multiple failed logon attempts followed by at least one successful logon attempt for each user and destination combination. This information can be valuable for detecting potential security breaches or suspicious activities related to user logon attempts.

## 7. Identifying High Network Bandwidth Consumption from a Baseline

### High Network Bandwidth Consumption:

```
➔ index=botsv2 sourcetype=pan:traffic earliest=1
| bin span=1d _time
| stats sum(bytes_out) AS bytes_out BY _time, src_ip, dest_ip
| eventstats avg(bytes_out) AS avg, stdev(bytes_out) AS stdev
| eval upper_bound = avg + 2 * stdev
| where bytes_out > upper_bound
| eval mbytes_out = bytes_out/1024/1024, upper_bound_mbytes =
upper_bound/1024/1024
| fields _time, src_ip, dest_ip, mbytes_out, upper_bound_mbytes
| search NOT dest_ip IN ("52.40.10.231", "23.218.119.50")
```

The screenshot shows the Splunk Search interface. At the top, there's a 'New Search' header with buttons for 'Save As', 'Create Table View', and 'Close'. Below this is a search bar containing the query: `index=botsv2 sourcetype=pan:traffic earliest=1 | bin span=1d _time | stats sum(bytes_out) AS bytes_out BY _time, src_ip, dest_ip | eventstats avg(bytes_out) AS avg, stdev(bytes_out) AS stdev | eval upper_bound = avg + 2 * stdev | where bytes_out > upper_bound | eval mbytes_out = bytes_out/1024/1024, upper_bound_mbytes = upper_bound/1024/1024 | fields _time, src_ip, dest_ip, mbytes_out, upper_bound_mbytes | search NOT dest_ip IN ("52.40.10.231", "23.218.119.50")`. Below the search bar, it indicates '1,862,616 events (before 4/2/24 9:21:24.692 AM)' and 'No Event Sampling'. The results are displayed in a table with columns: `_time`, `src_ip`, `dest_ip`, `mbytes_out`, and `upper_bound_mbytes`. The table shows three rows of data for different time periods and IP addresses.

| _time      | src_ip       | dest_ip      | mbytes_out         | upper_bound_mbytes |
|------------|--------------|--------------|--------------------|--------------------|
| 2017-08-24 | 160.153.91.7 | 71.39.18.125 | 43.04915237426758  | 17.745660999981016 |
| 2017-08-26 | 10.0.2.107   | 10.0.1.101   | 37.635586738586426 | 17.745660999981016 |
| 2017-08-26 | 160.153.91.7 | 71.39.18.125 | 27.388551712036133 | 17.745660999981016 |

| **bin span=1d \_time**: Bins the timestamps into daily intervals using a span of 1 day. This allows for aggregation of data on a daily basis

| **eventstats avg(bytes\_out) AS avg, stdev(bytes\_out) AS stdev**: Computes the average and standard deviation of outbound bytes across all events.

| **eval upper\_bound = avg + 2 \* stdev**: Calculates an upper bound for outbound bytes based on the average plus two times the standard deviation. This is a common statistical approach to identify outliers.

| **where bytes\_out > upper\_bound**: Filters the data to include only events where the outbound bytes exceed the calculated upper bound. This helps to identify unusually large traffic volumes.

this query aims to identify unusually high outbound traffic volumes from different source IP addresses to various destination IP addresses, potentially indicating anomalous or suspicious network activity.